



μCSS

Ein Node-basierter CSS- und Web-Grafik-Prozessor.

npm-Paket: `gulp-mu-css`

Handbuch

Version 2.5.1

2026-06-18

Inhalt

1 Einführung.....	5
1.1 Werkzeuge und Kosten.....	5
1.2 Draft-Workflow (Affinity + Watch).....	5
1.3 Einordnung: Warum µCSS?.....	6
2 Installation und Start.....	7
3 Anwendungsfälle.....	8
3.1 Benannte Eigenschaften (Variablen).....	8
3.2 Berechnungen und Ausdrücke.....	8
3.3 Mehrere Layouts über Skin-Manifeste.....	9
3.4 Automatische Sprite-Generierung.....	9
3.5 Bilder vorladen (Preload).....	10
3.6 Cursor mit Hotspot und Fallback.....	10
3.7 Automatische Bilderzeugung (media-Steps).....	10
4 microPS (µPS) — Bildgeneratoren.....	12
4.1 API-Übersicht.....	12
4.2 ButtonAndIconCreator.....	13
4.2.1 CreateByTopLayerSets — ein Bild pro Gruppe.....	13
4.3 ApplConMaker.....	14
4.4 SpriteAtlas und TileSheet.....	14
4.5 SequenceStrip und DSD-Format.....	14
4.5.1 DSD erzeugen (`CreateDsdFromImages`).....	15
4.5.2 PSD-Sequenzgruppen.....	15
4.6 Raster-Modell, I/O und Arbeitsblatt.....	16
4.7 Bildanpassungen und Masken.....	16
4.7.1 Masken.....	16
4.7.2 Auswahl-Engine.....	17
4.7.3 ApplyGamma.....	17
4.7.4 ApplyBrightnessContrast.....	17
4.7.5 ApplyHueSaturation.....	17
4.7.6 ApplyAdjustmentStack.....	18
4.8 Raster-Transformationen.....	18
4.9 ExtendScript-Einstieg (`MuPsDoc`).....	19
4.10 PSD-Compositor und Ebenenzusammenführung.....	19
4.10.1 Nachgebildete Fülloptionen (Layer Styles).....	19
4.10.2 Stilübertragung (ButtonAndIconCreator) vs. Ebenenverknüpfung.....	21
4.10.3 Deckkraft-Modell (drei getrennte Stellschrauben).....	22
4.10.4 Referenz-PSD und Adobe-Ground-Truth.....	22
4.10.5 Werkzeuge (µPS-Entwicklung).....	22
4.11 Draft-Workflow (Affinity + Watch).....	23
4.11.1 Zwischenschritte raw → final (`outputBase`).....	23
5 microAU (µAU) — Sound-Atlas.....	24
5.1 API-Übersicht.....	24
5.2 SoundAtlasMaker.....	24
5.2.1 Loop-Punkte im Dateinamen.....	24
5.2.2 JSON-Format.....	25
5.2.3 MP3-Eingaben.....	25
5.3 Anbindung an µCSS (Manifest `sounds`).....	25
5.3.1 Geplant: CSS-Sounds, Auto-Wiring & Handler-Overrides.....	25
6 microFT (µFT) — Icon-Font.....	26

6.1 Glyphen-Benennung.....	26
6.2 FontGenerator.....	26
6.3 API-Bausteine.....	27
6.4 Anbindung an µCSS.....	27
7 Vue und Komponenten-Co-Location.....	28
7.1 Ablauf.....	28
7.2 Namespaces und globale State-Klassen.....	28
7.3 Bestehende Vue-SFCs migrieren.....	28
8 Grundideen von µCSS.....	29
8.1 Das .µ.css-Format.....	29
8.2 Werte-Interpolation µ(Ausdruck).....	29
8.3 Direktiven -µ: Ausdruck.....	29
8.4 Das Skin-Manifest.....	30
8.5 JavaScript ohne Ersatzzeichen.....	30
9 Build-Ablauf und Gulp-Integration.....	31
9.1 Verzeichniskonventionen.....	31
9.2 Kompilierungs-Ablauf.....	31
9.3 Inkrementeller Build und Cache.....	31
9.4 Gulp-Tasks und Watch.....	32
10 Manifest-Referenz (DefineSkin).....	33
10.1 vars.....	33
10.2 helpers.....	33
10.3 cursors.....	33
10.4 media.....	33
10.5 imageFormat.....	34
10.6 sprites.....	35
10.7 minify.....	36
10.8 sounds.....	36
10.9 font.....	37
10.10 files.....	38
11 µ-Auswertungskontext (Referenz).....	39
11.1 Das \$-Objekt.....	39
11.2 Farb- und Wert-Funktionen.....	39
11.3 Regel- und Dokument-Methoden.....	39
11.4 Sprite().....	40
11.5 Cursor().....	40
11.6 Helper-Funktionen und this-Bindung.....	40
11.7 afterWork-Hooks.....	41
12 Arbeiten mit Farben.....	42
12.1 Farben definieren.....	42
12.2 Alpha-Werte.....	42
12.3 Farbberechnungen.....	42
13 Node-API.....	43
14 Fehlerdiagnostik.....	44
15 Migration von µCSS.....	45
16 Versionshistorie.....	46
17 Rechtliches.....	49
17.1 MIT-Lizenz (Originaltext).....	49
17.2 Marken.....	49
17.3 Verwendete Bibliotheken.....	49

1 Einführung

μCSS ist ein Node-Modul zur Verarbeitung von CSS-Dateien und zur Generierung von Web-Grafiken. Dieses Handbuch beschreibt die Version 2 — den Node-basierten Nachfolger des 2013 eingeführten, Adobe-Photoshop-basierten μCSS 1: Aus erweiterten Quell-Stylesheets (`.μ.css`) und den Medienquellen (z. B. PSD-Entwürfen) entstehen per Gulp die fertigen Skin-Dateien — Standard-CSS plus alle benötigten Bilder, Sprites, Cursor, Fonts und Sounds. Die Bilderzeugung übernimmt das Schwester-Modul μPS; Sound-Atlanten und Icon-Fonts die Module μAU und μFT — jeweils als eigenes Kapitel (*microPS*, *microAU*, *microFT*).

Da das μ-Zeichen in Paket- und Repository-Namen (npm, git) immer wieder Probleme bereitet, lauten die technischen Namen `gulp-mu-css` und `gulp-mu-ps` — μCSS und μPS sind die Anzeigenamen.

Die wichtigsten Eigenschaften von μCSS 2:

- Quell-Stylesheets bleiben **syntaktisch valides CSS** — Editoren, Linter und Diff-Werkzeuge funktionieren unverändert.
- **Benannte CSS-Eigenschaften** über Skin-Variablen (`$.name`).
- **Berechnung von CSS-Werten durch JavaScript-Ausdrücke** direkt im Stylesheet — ohne Ersatzzeichen wie «»`!`.
- **Mehrere Layouts (Skins)** aus denselben Quellen über Manifest-Dateien.
- **Automatische Sprite-Atlas-Generierung** inklusive Retina-Varianten (`@2x`) und `image-set()`.
- **Cursor-Verwaltung** mit Hotspot, Fallback und Retina-Unterstützung.
- **Vorladen von CSS-Bildern** über eine generierte Preload-Regel.
- **Automatische Bilderzeugung** aus PSD-Entwürfen (Buttons, Icons, App-Icons, Animations-Strips) via μPS.
- **Inkrementeller Build mit Cache** — nur Geändertes wird neu generiert.
- **Aussagekräftige Fehlermeldungen** mit Datei, Zeile und Quelltext-Ausschnitt.

Im Gegensatz zu μCSS 1 läuft Version 2 vollständig in Node.js (ab Version 18) und benötigt weder Photoshop noch andere Adobe-Produkte. Die Build-Steuerung erfolgt über Gulp oder direkt über die Node-API.

1.1 Werkzeuge und Kosten

Version 2 braucht **kein Adobe-Abo** für den Build: Sprites, Cursor und PSD-Rendering laufen headless über Node und μPS.

Geschichtete PSD-Entwürfe pflegt man üblicherweise in **[Affinity]**(<https://affinity.studio/download>) — der empfohlenen Alternative zu Photoshop. Die App ist **kostenlos** (Anmeldung mit kostenlosem Canva-Konto zum Download und zur Aktivierung). Als PSD exportieren oder speichern; μPS liest dieselbe Ebenenstruktur wie im Legacy-Workflow. Bei μCSS 1 hingen Kompilierung und Bitmap-Erzeugung an einer installierten, lizenzierten Photoshop-Kopie — laufende Kosten, die für viele Teams entfallen.

1.2 Draft-Workflow (Affinity + Watch)

Affinity ist nicht scriptfähig — Makros und Batch-Jobs lassen sich beim Öffnen oder Start nicht anstoßen. Die Automatisierung teilt sich deshalb klar:

Schritt	Werkzeug	Rolle
Authoring	Affinity (kostenlos)	Geschichtete PSD-Entwürfe pflegen

Schritt	Werkzeug	Rolle
Trigger	PSD speichern → WatchDrafts (μPS) oder gulp.watch	Entprellter Rebuild (~1,5 s)
Verarbeitung	μPS in Node	Ebenen lesen, Stile übertragen, compositen, PNG-Export
Optional	OpenDrafts (μPS)	Entwurf in Affinity öffnen nach fehlgeschlagenem Vergleich

μPS übernimmt die Ebenenlogik des alten Photoshop-Skripts; Affinity liefert nur die Quelldatei. API in `gulp-mu-ps`: `OpenDrafts`, `WatchDrafts`; CLI: `tools/open-drafts.mjs`, `tools/watch-drafts.mjs`. Details: `docs/CONCEPT.md` (D11).

1.3 Einordnung: Warum μCSS?

Für fast jeden Teilaspekt von μCSS existiert ein etabliertes Einzelwerkzeug — aber kein System, das alles bündelt:

Funktion	Nächstes existierendes Pendant	Was dort fehlt
Variablen & Farbfunktionen	Sass/LESS (<code>lighten()</code> , <code>mix()</code>), natives CSS (<code>color-mix()</code> , Custom Properties)	kein eingebettetes JavaScript
JavaScript in CSS-Werten	postcss-functions (registrierte JS-Funktionen als CSS-Funktionen)	nur benannte Funktionen — keine freien Ausdrücke, keine Direktiven mit Regel-/Dokument-Zugriff
Stylesheets mit voller JS-Power	vanilla-extract (Stylesheets-in-TypeScript)	umgekehrter Ansatz — das normale CSS-Format geht verloren
Sprite-Atlas aus CSS-Referenzen	spritesmith-Familie, postcss-sprites	weitgehend eingefroren, kein Retina-image-set-Workflow, kein gemeinsamer Cache
Bilderzeugung aus PSD-Entwürfen	nur Bausteine (<code>ag-psd</code> , Asset-Export aus Figma/Sketch)	kein Serien-Rendering mit Ebenenstil-Übertragung, nicht mit dem CSS-Build gekoppelt
Cursor, Preload, Skin-Manifest, Medien-Cache	handgestrickte Build-Skripte	nirgends gebündelt

Ein Teil der ursprünglichen μCSS-Motivation von 2013 ist heute durch natives CSS gelöst (Custom Properties, `color-mix()`, Nesting) — deshalb verzichtet μCSS 2 bewusst auf LESS-Unterstützung und Vendor-Prefixes. Der verbleibende Kern ist konkurrenzlos: beliebige JavaScript-Ausdrücke im CSS plus Direktiven mit AST-Zugriff, gekoppelt mit Sprite-Atlas inklusive Retina, PSD-Render-Pipeline und inkrementellem Cache, gesteuert über ein Manifest pro Skin. Und da μCSS intern eine PostCSS-Pipeline ist, bleibt das gesamte PostCSS-Ökosystem (`cssnano`, `Stylelint`, ...) andockbar, statt in Konkurrenz dazu zu stehen.

2 Installation und Start

μCSS und μPS sind als npm-Module `gulp-mu-css` und `gulp-mu-ps` verfügbar. Die Installation erfolgt im eigenen Projekt:

```
npm install gulp-mu-css gulp-mu-ps
```

μCSS hängt von μPS ab (Atlas- und Bilderzeugung) — nicht umgekehrt. Beide Module sind separat verwendbar.

Der typische Einstieg ist ein Gulp-Task, der ein Skin-Manifest baut:

```
// gulpfile.mjs
import gulp from "gulp";
import { BuildSkin } from "gulp-mu-css";

export async function SkinStd() {
  await BuildSkin("skins/src/std.μcss.mjs");
}
export function SkinWatch() {
  gulp.watch(["skins/src/**", "dev/media/final/**"], SkinStd);
}
```

Der Aufruf `npx gulp SkinStd` kompiliert den Skin „std“ in das Verzeichnis `skins/std/`. `BuildSkin` ist idempotent und cache-gestützt: Ein zweiter Aufruf ohne Änderungen ist nach wenigen Millisekunden fertig (siehe Kapitel „Build-Ablauf“).

3 Anwendungsfälle

Die folgenden Abschnitte zeigen die typischen Einsatzszenarien — analog zu den Use Cases des alten μ CSS-Handbuchs, aber in der neuen Syntax.

3.1 Benannte Eigenschaften (Variablen)

Der einfachste Anwendungsfall: Ein Wert wird einmal im Skin-Manifest benannt und an beliebig vielen Stellen verwendet. Im Manifest:

```
// skins/src/std.μcss.mjs
import { DefineSkin } from "gulp-mu-css";

export default DefineSkin({
  vars: {
    textColor: "#ff0000",
    backColor: "#0000ff"
  },
  files: [{ source: "src.μ.css", target: "std.css" }]
});
```

Im Quell-Stylesheet `src.μ.css`:

```
div.mydiv {
  padding: 5px;
  font-size: 24px;
  color: μ($.textColor);
  background-color: μ($.backColor);
  border: 10px μ($.backColor) solid;
  border-radius: 10px;
}
```

Kompiliert nach `std.css`:

```
div.mydiv {
  padding: 5px;
  font-size: 24px;
  color: #ff0000;
  background-color: #0000ff;
  border: 10px #0000ff solid;
  border-radius: 10px;
}
```

Anders als beim alten μ CSS sind keine Platzhalter-Properties und keine `Set*`-Direktiven mehr nötig: Der Wert steht direkt dort, wo er hingehört.

3.2 Berechnungen und Ausdrücke

Der Inhalt von `μ(...)` ist ein beliebiger JavaScript-Ausdruck. Damit sind Berechnungen, Bedingungen und Farboperationen direkt im Stylesheet möglich:

```
td.subheadline {
  border-bottom: 1px dashed μ(Lighten($.selectBaseBrgdColor, 0.7));
}
div.panel {
  padding: μ(Math.floor($.basePadding * 0.2))px;
  background-color: μ(Alpha($.baseBrgdColor, 0.85));
  z-index: μ($.PanelZIndex + 10);
}
div.hint {
  color: μ($.darkMode ? "#e0e0e0" : "#202020");
}
```


Für Operationen, die mehrere Properties erzeugen oder die Regel als Ganzes verändern, gibt es Direktiven (`-μ:`), zum Beispiel mit eigenen Makro-Funktionen aus dem Manifest:

```
div.menu {
  -μ: Borders($.menuBaseBrngdColor, 1, 0.3, -0.3, -0.3, 0.3);
}
```

Die Direktive ruft die Helper-Funktion `Borders` auf, die vier `border-*`-Properties berechnet und in die Regel einfügt (siehe Kapitel „`μ`-Auswertungskontext“).

3.3 Mehrere Layouts über Skin-Manifeste

Das alte Template-Compiling (eine Vorlagen-CSS plus je eine `μ.layout.css` pro Layout) wird durch mehrere Manifeste ersetzt, die sich dieselben `μ.css`-Quellen teilen:

```
// skins/src/layout_a.mjs
import { DefineSkin } from "gulp-mu-css";
export default DefineSkin({
  vars: { textColor: "#ff0000", backColor: "#0000ff" },
  files: [{ source: "template.μ.css", target: "layout_a.css" }]
});

// skins/src/layout_b.mjs
import { DefineSkin } from "gulp-mu-css";
export default DefineSkin({
  vars: { textColor: "#ff00ff", backColor: "#00ff00" },
  files: [{ source: "template.μ.css", target: "layout_b.css" }]
});
```

Jedes Manifest erzeugt ein eigenes Ausgabeverzeichnis (`skins/layout_a/`, `skins/layout_b/`) mit eigener CSS, eigenen Bildern und eigenem Build-Cache. Gemeinsame Variablen lassen sich als normales JavaScript-Modul importieren und in beiden Manifesten verwenden.

3.4 Automatische Sprite-Generierung

Eines der Highlights von `μCSS` (in Version 1 wie in Version 2) ist die automatische Erzeugung von Sprite-Atlanten. Die Direktive `Sprite(url)` registriert das Bild einer Regel für den Atlas:

```
div.loginbutton {
  display: inline-block;
  -μ: Sprite("imgs/aqua/but_login_normal.png");
}
div.loginbutton:hover {
  display: inline-block;
  -μ: Sprite("imgs/aqua/but_login_hover.png");
}
div.helpbutton {
  display: inline-block;
  -μ: Sprite("imgs/aqua/but_help_normal.png");
}
```

Beim Build werden alle registrierten Bilder in ein einziges Atlas-Bild gepackt (`imgs/sprites.png`, dazu `imgs/sprites@2x.png` aus den `@2x`-Quellbildern) und die Regeln umgeschrieben:

```
div.loginbutton {
  display: inline-block;
  background-image: url(imgs/sprites.png);
  background-image: image-set(url(imgs/sprites.png)1x, url(imgs/sprites@2x.png)2x);
  background-repeat: no-repeat;
  background-position: 0px 0px;
  width: 55px;
  height: 55px;
}
```

`width`, `height`, `background-position` und `background-repeat` werden automatisch gesetzt; vorhandene Deklarationen dieser Properties in der Regel werden ersetzt. Anders als beim alten μ CSS entfallen die Vendor-Prefix-Varianten (`-webkit-image-set` usw.) — alle relevanten Browser unterstützen `image-set()` seit Jahren unprefixed.

Identische Quellbilder teilen sich automatisch eine Atlas-Position (Duplikat-Reduktion). Der Atlas wird nur neu gepackt, wenn sich die Bildmenge oder ein Quellbild geändert hat.

3.5 Bilder vorladen (Preload)

Wichtige Bilder (z. B. Hover-Zustände und Cursor) können beim Laden der Seite vorab geladen werden. μ CSS sammelt die Bild-URLs und erzeugt eine Preload-Regel, wenn die Manifest-Option `sprites.preloadRule` gesetzt ist:

```
div.csspreload {
    background-image: url(imgs/sprites.png), url(imgs/general/gui/cursors/zoom.png);
    display: none;
}
```

In der HTML-Seite genügt ein leeres Element mit dieser Klasse:

```
<div class="csspreload"></div>
```

Cursor-Bilder aus den `cursors`-Definitionen werden automatisch in die Preload-Liste aufgenommen.

3.6 Cursor mit Hotspot und Fallback

Cursor werden im Manifest definiert und im Stylesheet per Name verwendet. Definition:

```
cursors: [
    { name: "zoom", fallback: "zoom-in", image: "imgs/general/gui/cursors/zoom.png",
      hotspot: [10, 8] },
    { name: "wait", fallback: "wait", image: "imgs/general/gui/cursors/wait.png", hotspot:
      [12, 12] }
]
```

Verwendung als Direktive oder als Wertform:

```
*.cursor_zoom {
    -μ: Cursor("zoom");
}
a.preview {
    cursor: μ(Cursor("zoom"));
}
```

Die Direktive erzeugt die `url()`-Form mit Hotspot und Fallback sowie — wenn eine `@2x`-Bilddatei existiert — zusätzlich die `image-set()`-Variante:

```
*.cursor_zoom {
    cursor: url(imgs/general/gui/cursors/zoom.png) 10 8, zoom-in;
    cursor: image-set(url(imgs/general/gui/cursors/zoom.png)1x,
url(imgs/general/gui/cursors/zoom@2x.png)2x) 10 8, zoom-in;
}
```

3.7 Automatische Bilderzeugung (media-Steps)

Was früher die μ CSS-Plugins (ButtonCreator, ApplIconMaker, FileCopy) erledigt haben, übernehmen jetzt die `media`-Einträge im Manifest. Sie laufen vor der CSS-Kompilierung und erzeugen bzw. kopieren alle Medien in das Skin-Verzeichnis:

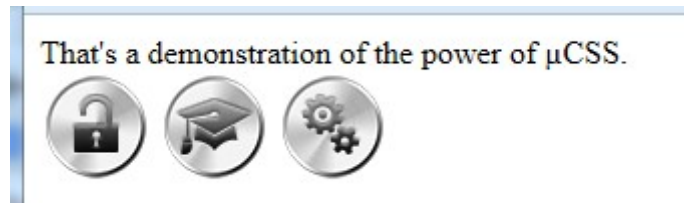
```
media: [
    { buttonsAndIcons: "dev/media/final/general/gui/panelbuttons.psd", layout: "std",
outputDir: "imgs/general/gui/panelbuttons" },
]
```

```

{ buttonsAndIcons: "dev/media/final/general/gui/cursors.psd", layout: "std", outputDir:
"imgs/general/gui/cursors" },
{ appIcons: "dev/media/final/favicon.psd", layout: "std", profiles: ["web"] },
{ sequenceStrip: "dev/media/raw/glittery/imgs", outputFile:
"imgs/general/gui/glittery/glittery.png" },
{ copy: "dev/media/final/general/gui/teaserbgrd.png", to: "imgs/general/gui" },
{ copyFolder: "dev/media/final/fonts", to: "fonts" }
]

```

- **buttonsAndIcons** rendert aus einem geschichteten PSD-Entwurf (Layouts × Icon-Glyphen) komplette Button- und Icon-Serien inklusive @2x-Varianten — der Nachfolger des ButtonCreator-Plugins. Die Fülloptionen (Schlagschatten, Verläufe, Schein, Relief, **Kontur**, **Glanz** usw.) werden von μPS nachgebildet — siehe Kapitel *PSD-Compositor*.



Dieselben Icon-Glyphen, gerendert mit zwei Layouts desselben Entwurfsdokuments („alu“ und „aqua“)



Mit mode: "topLayerSets" arbeitet der Step im zweiten Legacy-Modus (CreateByTopLayerSets): Statt der Icon×State-Matrix wird **jede direkte Untergruppe der Layout-Gruppe zu genau einem Bild**, benannt nach dem Gruppennamen. Eine icons-Gruppe ist hier nicht nötig — typisch für Logos, Animationsframes (z. B. activityindicator) und Emoticons. Optional grenzt setPattern (Regex-String) die exportierten Gruppen ein.

```

{ buttonsAndIcons: "dev/media/final/general/activityindicator.psd", layout: "std",
mode: "topLayerSets", outputDir: "imgs/general" }

```

- **appIcons** erzeugt App-Icons und Favicons profilbasiert (web, ios, play) aus einem quadratischen Master — der modernisierte ApplconMaker.
- **sequenceStrip** baut aus einer Bildsequenz (Einzeldateien oder ein Bild im DSD-Format) einen horizontalen Animations-Strip mit JSON-Frame-Daten:



Animations-Strip, generiert aus 19 Einzel-Frames einer Rauch-Animation

- **copy / copyFolder** kopieren statische Dateien (Make-artig: nur wenn die Quelle neuer ist).

Ausführliche Referenz aller Bildgeneratoren, DSD-Format, Draft-Workflow und Raster-APIs: **Kapitel [microPS (μPS)](#microps-μps--bildgeneratoren)**.

4 microPS (μPS) — Bildgeneratoren

Das Schwester-Modul **μPS** (`gulp-mu-ps`, Anzeigename **μPS**) übernimmt die bildverarbeitenden Funktionen des alten Adobe-Workflows — **ohne Adobe-Abhängigkeit**. μCSS bindet μPS über `media-steps` und `Sprite-/Cursor-Direktiven` ein; μPS ist auch **standalone** per `npm install gulp-mu-ps` nutzbar.

PSD-Quellen liest μPS über `ag-psd`, Bildverarbeitung läuft über `sharp`. Geschichtete Entwürfe pflegt man üblicherweise in **[Affinity](https://affinity.studio/download)** (kostenlos; Canva-Konto zum Download). Die wichtigsten Fülloptionen sind nachgebildet (Schlagschatten, Verläufe, Schein, Relief, **Kontur, Glanz**); Musterüberlagerung und exakte Verlauf-Konturen fehlen — Details im Kapitel **PSD-Compositor**.

4.1 API-Übersicht

Export	Beschreibung
<code>ButtonAndIconCreator</code>	Button-/Icon-Serien aus PSD-Entwürfen (Layouts × Glyphen)
<code>AppIconMaker</code>	App-Icons/Favicons profilbasiert (<code>web</code> , <code>ios</code> , <code>play</code>)
<code>SpriteAtlas</code>	Sprite-Atlas mit Duplikat-Reduktion und JSON-Map
<code>TileSheet</code>	Tile-Textur: Zerlegung, Dedupe, leere Tiles
<code>SequenceStrip</code>	Horizontale Animations-Strips aus Dateiserien oder DSD-Bildern
<code>ScanDsdImage</code>	DSD-Scanner (Zellen, Typ, Frame-Nummer, Anker)
<code>CreateDsdFromImages</code>	DSD-Master aus Frame-Bildern erzeugen
<code>CreatePsdWithSequence</code> , <code>InsertSequenceIntoGroup</code>	Frame-Sequenz in PSD-Ebenengruppe schreiben
<code>ApplyGamma</code> , <code>ApplyBrightnessContrast</code> , <code>ApplyHueSaturation</code> , <code>ApplyAdjustmentStack</code>	PS-ähnliche Anpassungen mit optionaler Maske
<code>MaskFromRects</code> , <code>VisitMaskedPixels</code>	Masken-Helfer (z. B. Kachel-Auswahlen)
<code>MaskFromAlpha</code> , <code>MagicWand</code> , <code>ColorRange</code> , <code>FeatherMask</code> , <code>InvertMask</code> , <code>SelectionBounds</code> , <code>CopySelection</code>	Auswahl-Engine (Alpha, Zauberstab, Farbbereich, Weiche, Invert)
<code>CreateRaster</code> , <code>CloneRaster</code>	Leeres Raster / Kopie
<code>LoadRasterFromImage</code> , <code>SaveRasterAsImage</code> , <code>SaveRasterAsImageHalfSize</code>	Bild laden/speichern
<code>ScaleRaster</code> , <code>CropRaster</code> , <code>FlipRaster</code> , <code>RotateRaster</code> , <code>RotateRasterAround</code> , <code>PasteRaster</code> , <code>ResizeCanvas</code>	Geometrie und Arbeitsblattgröße
<code>MuPsDoc</code>	ExtendScript-ähnlicher Einstieg (<code>flat</code> , Phase 1)
<code>OpenDrafts</code> , <code>WatchDrafts</code>	Affinity-/Draft-Workflow (speichern → Rebuild)
<code>PsdDocument</code> , <code>RenderDocument</code>	PSD laden und compositen

4.2 ButtonAndIconCreator

Erwartete Ebenenstruktur (Master @2x):

```
layouts/  
  <layout>/          z. B. "std", "alu", "aqua"  
  <state>/           z. B. "normal", "hover" – oder "_" (siehe Namensschema)  
  ...                Hintergrundebenen  
  icon               Platzhalter mit Fülloptionen für Icons  
icons/  
  <glyphName>        je Glyphe eine Pixel-Ebene oder Gruppe, z. B. "but_login_"
```

Für jede Kombination aus Glyphe und State werden die **Fülloptionen** (effects) des icon-Platzhalters auf die Glyphe übertragen (Copy/Paste Layer Style) — **nicht** der Platzhalter-Blend-Modus oder die Ebenen-Deckkraft; das Dokument wird gerendert und gespeichert. Glyphen namens `_` werden übersprungen.

```
import { ButtonAndIconCreator } from "gulp-mu-ps";  
  
await ButtonAndIconCreator.Create("drafts/buttons.psd", {  
  layout: "aqua",  
  outputDir: "imgs/aqua"  
});
```

Option	Default	Wirkung
layout	— (Pflicht)	Layout-Gruppe unter layouts/; der Name erscheint nicht im Dateinamen — Unterscheidung über outputDir.
outputDir	— (Pflicht)	Zielverzeichnis.
retina	true	@2x-Master: <name>@2x.<ext> und <name>.<ext> (50%-Downscale, Lanczos). Mit false nur eine Datei in Dokumentauflösung.
format	"png"	"png" oder "webp".
iconsGroupName	"icons"	Gruppe der Glyphen.
layoutsGroupName	"layouts"	Gruppe der Layouts.

Dateinamen: <glyphName><stateName>[@2x].<ext> — State wird **ohne Trennzeichen** angehängt (but_login_ + normal → but_login_normal.png). State `_` → kein Suffix (pointer.png). Layout-Namen nie im Dateinamen.

4.2.1 CreateByTopLayerSets — ein Bild pro Gruppe

Keine icons-Gruppe nötig: jede direkte Untergruppe des Layouts ist eine fertige Komposition → ein Bild, benannt nach der Gruppe (Logos, Animationsframes, Emoticons).

```
<layout>/  
  <setName>/          z. B. "frame01"  
  ...                übersprungen  
  _/
```

```
await ButtonAndIconCreator.CreateByTopLayerSets("drafts/activityindicator.psd", {  
  layout: "std",  
  outputDir: "imgs/general",  
  setPattern: /^frame/ // optional: nur passende Gruppen
```

```
});
```

Beim Rendern ist nur die aktuelle Untergruppe sichtbar; Geschwister werden ausgeblendet — wie im Legacy-Plugin.

4.3 AppIconMaker

Quadratischer Master (PSD/PNG, $\geq 1024 \times 1024$):

```
import { AppIconMaker } from "gulp-mu-ps";

await AppIconMaker.Create("drafts/appicon.psd", {
  outputDir: "icons",
  profiles: ["web", "ios", "play"],
  layout: "aqua",
  appName: "MyApp",
  themeColor: "#0a5ae0",
  background: "#ffffff"
});
```

Profil	Erzeugt
web	favicon.ico, PWA-Icons, apple-touch-icon, site.webmanifest, HTML-Snippet
ios	appstore-icon-1024.png (ohne Alpha)
play	Play-Store-Icon plus adaptive Foreground/Background-Layer

Custom-Profile: { name, icons: [{ file, size, mode }] } mit mode: plain, flatten, maskable, background, ico.

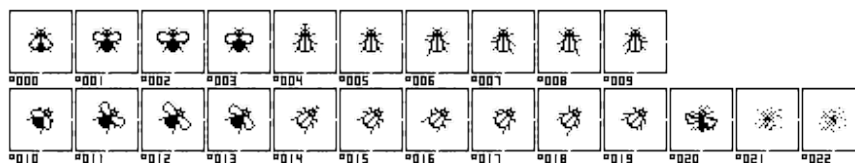
4.4 SpriteAtlas und TileSheet

SpriteAtlas packt viele Einzelbilder in einen Atlas (1x/@2x, optional WebP, JSON-Map). TileSheet zerlegt Quellen in uniforme Tiles, dedupliziert und schreibt quadratische Texturen plus JSON-Map — Basis für Oxyd/Unity-Pipelines (App-spezifische Nachbearbeitung siehe docs/CONCEPT.md D12).

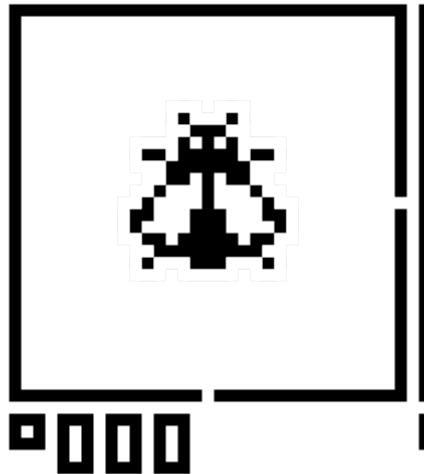
4.5 SequenceStrip und DSD-Format

SequenceStrip baut aus Einzeldateien oder einem **DSD-Bild** einen horizontalen Strip plus JSON-Frame-Daten (Legacy-SpriteTools-kompatibel).

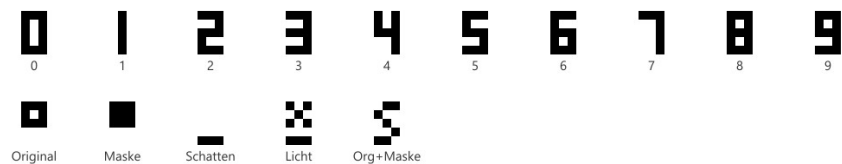
DSD („Dongleware Sprite Definition“): alle Frames in **einem** Bild, jeder Frame in einer markierten Zelle mit Anker-Markern und Label (Typ-Glyphe + dreistellige Frame-Nummer):



DSD-Quellbild (flyex)



DSD-Zelle vergrößert



DSD-Glyphen-Referenz

Der Scanner `ScanDsdImage` erkennt Zellen automatisch; `SequenceStrip` sortiert nach Frame-Nummer:



Getrimmte Inhaltsbreite beim Erzeugen ≥ 10 px, damit das Frame-Label in den Zellrahmen passt.

4.5.1 DSD erzeugen (`CreateDsdFromImages``)

Gegenstück zu `ScanDsdImage` — aus Frame-PNGs ein DSD-Masterblatt:

```
import { CreateDsdFromImages, DSD_SIGN_ORIGINAL } from "gulp-mu-ps";

await CreateDsdFromImages(["frames/f0.png", "frames/f1.png"], {
  outputFile: "sprites/series.png",
  signType: DSD_SIGN_ORIGINAL,
  pivotPercent: { x: 0, y: 0 }, // Anker links oben am getrimmten Inhalt (Default)
  frameNumbers: [0, 1]
});
```

Pivot: `pivotPercent` (0...100 relativ zum getrimmten Inhaltsrechteck) ist die empfohlene Angabe — 0/0 = links oben, 50/50 = Mitte, 100/100 = rechts unten. Alternativ `pivot: { x, y }` in **Pixel-Offsets** (DSD-Konvention, negativ = Anker rechts/unten vom Inhalt). Pro Frame wird der Anker aus der jeweiligen Trim-Box berechnet.

4.5.2 PSD-Sequenzgruppen

```
import { CreatePsdWithSequence, InsertSequenceIntoGroup } from "gulp-mu-ps";

await CreatePsdWithSequence(["frames/f0.png", "frames/f1.png"], {
  outputFile: "drafts/sequence.psd",
  groupName: "animation"
});
```

`PsDocument` unterstützt `Save()` für Round-Trips nach In-Memory-Änderungen.

4.6 Raster-Modell, I/O und Arbeitsblatt

µPS arbeitet intern mit **Rastern**: `{ width, height, data: Float32Array }` — **RGBA 0...1**, nicht premultiplied, volle Dokumentfläche.

Export	Beschreibung
<code>CreateRaster(w, h)</code>	Leeres Arbeitsblatt (transparent)
<code>CloneRaster(raster)</code>	Tiefe Kopie
<code>LoadRasterFromImage(path)</code>	PNG/WebP/JPEG/... laden
<code>SaveRasterAsImage(raster, path)</code>	Speichern (.png / .webp per Endung)
<code>SaveRasterAsImageHalfSize(raster, path)</code>	@2x → 1x (Lanczos 50 %)

```
import { CreateRaster, LoadRasterFromImage, SaveRasterAsImage } from "gulp-mu-ps";

const blank = CreateRaster(512, 512);
const photo = await LoadRasterFromImage("photo.jpg");
await SaveRasterAsImage(photo, "out/photo.webp");
```

Arbeitsblattgröße ändern (Inhalt **nicht** skalieren — PS *Bild → Arbeitsfläche*):

```
import { LoadRasterFromImage, ResizeCanvas, SaveRasterAsImage } from "gulp-mu-ps";

const raster = await LoadRasterFromImage("icon.png");
const padded = ResizeCanvas(raster, {
  width: 128,
  height: 128,
  anchor: "middleCenter",
  fill: [0, 0, 0, 0]
});
await SaveRasterAsImage(padded, "out/icon-padded.png");
```

Anker: `topLeft`, `topCenter`, `topRight`, `middleLeft`, `middleCenter`, `middleRight`, `bottomLeft`, `bottomCenter`, `bottomRight`. Beim Verkleinern wird überstehender Inhalt abgeschnitten.

Geplant (Baseline, noch nicht implementiert): Indexed-Farbraum / **CLUT** und **Bit-Tiefen-Reduktion** (noofbits) für Textur-Pipelines — siehe `docs/CONCEPT.md` D15.

4.7 Bildanpassungen und Masken

Headless-Pendant zu PS *Bild → Anpassungen*. Alle Funktionen ändern das Raster **in-place**; optional nur innerhalb einer **Auswahl** (Maske).

4.7.1 Masken

Form	Verwendung
Rechteckliste <code>[{ x, y, w, h }, ...]</code>	Oxyd-Style: viele 64×64-Tiles als eine Auswahl
<code>MaskFromRects(w, h, rects)</code>	Gewichtsmask 0/1
<code>Float32Array</code> (Länge = <code>w×h</code>)	Weiche Maske 0...1
<code>undefined</code> / weggelassen	Ganzes Bild (alle nicht-transparenten Pixel)

4.7.2 Auswahl-Engine

Low-Level-Helfer für PS-ähnliche Auswahlen auf flachen RGBA-Rastern (RGB-Distanz 0...1):

Export	Wirkung
<code>MaskFromAlpha(raster, { min })</code>	Alle Pixel mit <code>Alpha ≥ min</code>
<code>MagicWand(raster, x, y, { tolerance })</code>	Verbundene Flood-Fill-Auswahl ab Seed-Pixel
<code>ColorRange(raster, { rgb, fuzziness })</code>	Global alle passenden Pixel (nicht nur verbunden)
<code>FeatherMask(mask, w, h, radius)</code>	Weiche Kanten (Box-Blur-Näherung)
<code>InvertMask(mask)</code>	Auswahl invertieren
<code>SelectionBounds(mask, w, h)</code>	Begrenzungsrechteck oder <code>null</code>
<code>CopySelection(raster, mask?)</code>	Ausgewählte Pixel in neues Raster kopieren

Über `MuPsDoc.selection`: `SelectAlpha`, `MagicWand`, `ColorRange`, `Feather`, `Invert`, `Bounds` — sowie `Copy()`, `Paste(other, { x, y })`, `Duplicate()` am Dokument.

4.7.3 ApplyGamma

PS *Belichtung* / Oxyd `GammCorrection(γ)`: Exponent $1/\gamma$ auf RGB.

Parameter	Bereich	Default
<code>_gamma</code>	~0.01...9.99 (1 = keine Änderung)	—
<code>exposure</code>	PS-Exposure	0
<code>offset</code>	PS-Offset	0
<code>mask</code>	siehe oben	ganzes Bild

4.7.4 ApplyBrightnessContrast

Parameter	Bereich	Default
<code>brightness</code>	−100...100	0
<code>contrast</code>	−100...100	0
<code>useLegacy</code>	lineares Modell (Alt-Skripte)	<code>false</code>

4.7.5 ApplyHueSaturation

Master-Kanal (keine Einzelfarbkanäle):

Parameter	Bereich	Default
<code>hue</code>	−180...180 (Grad)	0
<code>saturation</code>	−100...100	0
<code>lightness</code>	−100...100	0

4.7.6 ApplyAdjustmentStack

Mehrere Steps in Reihenfolge, **eine** Maske für alle Steps:

```
import {
  CreateRaster,
  ApplyGamma,
  ApplyAdjustmentStack,
  MaskFromRects
} from "gulp-mu-ps";

const raster = CreateRaster(512, 512);
const mask = MaskFromRects(512, 512, [
  { x: 0, y: 0, w: 64, h: 64 },
  { x: 64, y: 0, w: 64, h: 64 }
]);
ApplyAdjustmentStack(raster, [
  { type: "gamma", gamma: 1.2 },
  { type: "brightnessContrast", brightness: 10, contrast: 5 },
  { type: "hueSaturation", saturation: -15 }
], { mask });
```

Visuelle Nähe zu PS, keine Bit-Genauigkeit — gegen Referenz-PNGs tunen, wo nötig.

4.8 Raster-Transformationen

Geometrie auf dem Raster — Skalierung nutzt `sharp`-Kernel.

Export	Wirkung
<code>ScaleRaster</code>	Skalieren: <code>scale</code> , oder <code>width/height</code> , <code>kernel</code>
<code>CropRaster</code>	Rechteck { <code>x</code> , <code>y</code> , <code>w</code> , <code>h</code> } (an Bounds geklemmt)
<code>FlipRaster</code>	"horizontal" / "vertical"
<code>RotateRaster</code>	90°, 180°, 270°
<code>RotateRasterAround</code>	Freier Winkel, Pivot { <code>x</code> , <code>y</code> } (bilinear)
<code>PasteRaster(cw, ch, raster, { x, y })</code>	Raster auf größeres leeres Blatt legen
<code>ResizeCanvas</code>	Arbeitsblattgröße (siehe oben)

```
import {
  LoadRasterFromImage,
  ScaleRaster,
  CropRaster,
  FlipRaster,
  RotateRasterAround,
  SaveRasterAsImage
} from "gulp-mu-ps";

const raster = await LoadRasterFromImage("photo.jpg");
const half = await ScaleRaster(raster, { scale: 0.5, kernel: "lanczos3" });
const patch = CropRaster(half, { x: 100, y: 50, w: 640, h: 320 });
const tilted = RotateRasterAround(patch, 15, { x: 0, y: 0 });
await SaveRasterAsImage(tilted, "out/patch.png");
```

ScaleRaster-Kernel: `nearest`, `linear`, `cubic`, `lanczos2`, `lanczos3` (Default). Näherung an PS
`ResampleMethod: BICUBICSHARPER` → `lanczos3`, `BILINEAR` → `linear`, `NEARESTNEIGHBOR` → `nearest`.

Visueller Vergleich: `node gulp-mu-ps/tools/verify-image-ops.mjs <bild> <ausgabeordner>`.

4.9 ExtendScript-Einstieg (`MuPsDoc`)

Optionaler **Kompatibilitäts-Wrapper** für JSX-/Oxyd-Gewohnheiten — **kein** PS-DOM, keine executeAction-Emulation. Mappt vertraute Namen auf die Raster-Engine (Phase 1: flache Bilder).

ExtendScript / Oxyd	MuPsDoc / μ PS
app.activeDocument (flat)	await MuPsDoc.Open(path)
doc.width / doc.height	.width / .height
selection.select(rects)	doc.selection.Select([...])
selection.selectAlpha / Zauberstab / Farbbereich	SelectAlpha, MagicWand(x,y), ColorRange({ rgb, fuzziness }), Feather, Invert, Bounds
GammCorrection(γ)	doc.GammaCorrection(γ)
Helligkeit/Kontrast	doc.BrightnessContrast(b, c)
Farbton/Sättigung	doc.HueSaturation({ ... })
doc.resizeImage(...)	await doc.ResizeImage({ scalePercent, method: "BICUBICSHARPER" })
Arbeitsfläche	doc.CanvasSize({ width, height, anchor })
doc.saveAs(...)	await doc.SaveAs(path)
@2x + 50%-Downscale	await doc.SaveAsRetinaPair(dir, name)
Copy / Paste / Duplicate	doc.Copy(), doc.Paste(other, { x, y }), doc.Duplicate()

```
import { MuPsDoc } from "gulp-mu-ps";

const doc = await MuPsDoc.Open("atlas.png");
doc.selection.MagicWand(12, 8, { tolerance: 0.08 });
doc.GammaCorrection(1.2);
const patch = doc.Copy();
await doc.SaveAs("out/atlas-gamma.png");
```

Phase 2 (geplant): PSD (OpenPsd), findLayer, copyLayerStyle, flatten — siehe docs/CONCEPT.md D14.

Die **Low-Level-API** (ApplyGamma, ScaleRaster, ...) bleibt die Engine; MuPsDoc ist nur Einstiegshilfe für Migration.

4.10 PSD-Compositor und Ebenenzusammenführung

Unter der API-Oberfläche compositen **PsDocument** / **RenderDocument** und **Compositor.mjs** geladene PSDs: sichtbare Ebenen, Gruppen, Text, Fülloptionen, Blend-Modi. Das ist die Grundlage für **ButtonAndIconCreator** (Stilübertragung) und direktes Rendering.

Visuelle Nähe zu Photoshop/Affinity, keine Bit-Genauigkeit — Abweichungen werden per **MAE**-Regression gegen Adobe-Referenz-PNGs überwacht (test/ReferenceRender.test.mjs).

4.10.1 Nachgebildete Fülloptionen (Layer Styles)

μ PS liest Fülloptionen aus der PSD (ag-psd, Feld effects) und rendert sie in **Effects.mjs**. Visuelle Nähe zu Photoshop/Affinity, **keine Bit-Genauigkeit** — Abweichungen werden per MAE-Regression gegen Adobe-Referenz-PNGs überwacht.

Unterstützt

Photoshop / Affinity (DE)	Schlüssel (ag-psd)	Anmerkung
Farbüberlagerung	solidFill	inkl. Deckkraft und Blend-Modus pro Effekt
Verlaufsüberlagerung	gradientOverlay	nur linear (Winkel aus PSD); radial/winkel/reflektierend nicht
Schein innen	innerGlow	Größe, Spread (Choke), Umfang (Range), Konturkurve
Schein außen	outerGlow	wie innen; nur außerhalb der Form sichtbar
Innen-/Schlagschatten	innerShadow, dropShadow	Abstand, Winkel, Größe, Spread, Farbe, Blend-Modus
Relief / Prägung	bevel	Stile: inner bevel, outer bevel, pillow emboss, emboss; Technik smooth vs. chisel hard; Konturkurve auf Relief; getrennte Highlight-/Schatten-Farben und Blend-Modi
Kontur (Stroke)	stroke	Position inside / center / outside; Farbe (und einfacher Verlauf via erste Farbe); kein Muster-Kontur
Glanz (Satin)	satin	Abstand, Größe, Winkel, Invert, Konturkurve, Blend-Modus

Konturkurve vs. Kontur-Effekt: In den Effekt-Dialogen heißt der Tab *Kontur* die **Verlaufskurve** (Falloff) von Schein, Relief und Glanz — die wird ausgelesen und angewendet. Der separate Layer-Style **Kontur** (Stroke / Umrisslinie) ist oben als stroke aufgeführt.

Nicht implementiert (werden ignoriert)

Photoshop / Affinity (DE)	Schlüssel	Empfehlung
Musterüberlagerung	patternOverlay	Muster als Pixel-Ebene (ag-psd: Patt-Sektion eingeschränkt)
Kontur mit Musterfüllung	stroke (fillType: pattern)	Farb-Kontur oder Pixel-Ebene
Weitere Verlaufstypen	—	nur linear in gradientOverlay; Kontur-Verlauf nur Näherung (erste Farbe)

Blend-Modi (Ebenen und Effekte)

Jede **Ebene** und jeder **einzelne Effekt** kann einen eigenen Blend-Modus tragen (PS-Dialog „Modus“ / „Ebenenverknüpfungsmodus“ am Effekt). µPS wendet dieselbe Tabelle in **BlendModes.mjs** an — beim Compositing der Ebene und beim Mischen von Effekt-Farben (z. B. innerer Schatten mit *Farbig abwedeln*).

Englisch (ag-psd)	Deutsch (Photoshop)	µPS
normal	Normal	✓

Englisch (ag-psd)	Deutsch (Photoshop)	µPS
multiply	Multiplizieren	✓
screen	Abwedeln	✓
overlay	Überlagern	✓
darken	Abdunkeln	✓
lighten	Aufhellen	✓
color burn	Farbig nachbelichten	✓
color dodge	Farbig abwedeln	✓
linear burn	Linear abwedeln (oft „Negativ multiplizieren“)	✓
linear dodge	Linear abwedeln (Addieren)	✓
hard light	Hartes Licht	✓
soft light	Weiches Licht	✓
difference	Differenz	✓
exclusion	Ausschluss	✓
pass through	Durchreichen (Gruppen)	✓

Nicht unterstützt (Fallback: Normal): u. a. *Abdunkeln 2*, *Aufhellen 2*, *Linear abwedeln 2*, *Spitzlicht*, *Lineares Spitzlicht*, *Pin-Licht*, *Hart mischen*, *Farbe*, *Sättigung*, *Luminanz*, *Teilen*, *Subtrahieren*.

Referenz-Matrix in der PSD: `layouts/blend/` → PNGs unter `examples/reference/out/blend/` (normal, multiply, screen, overlay, darken, lighten, color burn).

4.10.2 Stilübertragung (ButtonAndIconCreator) vs. Ebenenverknüpfung

Zwei verschiedene Konzepte — im Handbuch oft verwechselt:

Mechanismus	Wo	Was passiert
Stil kopieren (CpFX/PaFX)	ButtonAndIconCreator	Der komplette effects-Block des Platzhalters <code>icon</code> wird auf die Glyphe gelegt (Legacy-Photoshop: Copy/Paste Layer Style).
Ebenenverknüpfung (PS *Link Layers*)	Authoring in PS/Affinity	Nur gemeinsames Verschieben im Editor — kein eigener Render-Modus in µPS.

Was Stilübertragung nicht kopiert (wie im Legacy-Plugin): **Blend-Modus**, **Ebenen-Deckkraft** und **Position** des Platzhalters — die Glyphe behält ihre eigenen Werte. Effekt-interne Blend-Modi (z. B. innerer Schatten → *Farbig nachbelichten*) liegen im **effects**-Objekt und **werden** übernommen.

Die Referenz-PSD enthält unter `layouts/links/` eine Demo mit verknüpften Ebenen (`icon_master`, `follower`) — zum visuellen Abgleich beim flachen Compositing, nicht als separates „Verknüpfungsmodus“-Feature.

4.10.3 Deckkraft-Modell (drei getrennte Stellschrauben)

Steuerung	PSD-Quelle	Wirkung in μ PS
Pixel-Alpha	Alpha pro Pixel in imageData	Formmaske für Effekte + Compositing-Alpha
Füll-Deckkraft	fillOpacity (iOpa), 0...1	Skaliert nur die Fill-Alpha; Fülloptionen bleiben voll
Ebenen-Deckkraft	opacity, 0...1	Skaliert die gesamte Ebene (Fill + Styles) beim Compositing

Regenbogen-Discs (statt Einfarbig) in `compositing/` decken Kanal-/Blend-Fehler auf, die flache Füllungen verbergen. Synthetische Tests: `test/Compositing.test.mjs`.

4.10.4 Referenz-PSD und Adobe-Ground-Truth

Für Entwicklung und Tuning liegt unter `gulp-mu-ps/examples/reference/` eine umfassende Referenz:

Pfad	Rolle
<code>examples/drafts/mups-reference.psd</code>	Referenz-PSD (Compositor, Effekte, Blend, Text, beide ButtonAndIconCreator-Modi)
<code>examples/reference/out/</code>	Adobe-Export-PNGs (Ground Truth)
<code>tools/photoshop/build-mups-reference.jsx</code>	PSD erzeugen
<code>tools/photoshop/export-mups-reference.jsx</code>	PNGs exportieren (Wiederholungen überspringen vorhandene Dateien)
<code>tools/build-reference-overview.mjs</code>	Kontaktblatt <code>out/overview/mups-reference-sheet.png</code>

Isolierte Effekt-Layouts (`layouts/fx/`): je ein Preset pro Layer-Style-Variante — u. a. `solidFill`, `dropShadow` (3 Varianten), `innerShadow` (4), `innerGlow/outerGlow` (je 3), `gradientOverlay` (3), `strokeOutside`, `strokeInside`, `strokeCenter`, `strokeMultiply`, `satin`, `satin_warm`, `satin_invert`, `bevelInner` (5 Relief-Varianten). Pro Layout entstehen Adobe-PNGs unter `out/fx/` (Stilübertragung auf `glyph_disc`) und `out/flat/fx/` (direktes Compositing ohne CpFX/PaFX).

Tuning-Hinweise: Kombinierte Stacks (`stacks/stack_aquaIcon`) für produktionsnahe Abgleiche; isolierte Paare für Parametersensitivität — z. B. `dropShadow` vs. `dropShadow_soft`, `bevelInner` vs. `bevelInner_chisel`, `strokeOutside` vs. `strokeMultiply`, `satin` vs. `satin_invert`. Nach Änderungen an `Effects.mjs` oder an den JSX-Presets: PSD neu bauen → exportieren → `npx gulp reference:render` → `npm test` (μ PS).

Workflow (Entwickler): `buttons.psd` → JSX bauen → exportieren → μ PS mit `retina: false` rendern → `compare-images.mjs` oder `ReferenceRender-Tests`. Details: `gulp-mu-ps/examples/reference/README.md`.

Nicht in dieser PSD: `SpriteAtlas`, `TileSheet`, `SequenceStrip/DSD`, `AppIconMaker` — eigene Legacy-Referenzen unter `oldsrcs/.../examples/` (PNG-/Sequenz-Pipelines).

4.10.5 Werkzeuge (μ PS-Entwicklung)

- `node tools/inspect-psd.mjs <file.psd>` — Ebenenbaum und Fülloptionen
- `node tools/compare-images.mjs <a> <b>` — PNG-Vergleich (MAE)
- `node tools/open-drafts.mjs / watch-drafts.mjs` — Draft-Workflow aus der Shell

4.11 Draft-Workflow (Affinity + Watch)

Affinity hat **keine Script-API** — der Automatisierungspfad:

1. PSD in Affinity bearbeiten und speichern.
2. WatchDrafts oder gulp.watch erkennt die Änderung (Debounce ~1,5 s).
3. µPS rendert headless; µCSS-Build läuft bei Bedarf mit.

```
import { WatchDrafts, OpenDrafts } from "gulp-mu-ps";

WatchDrafts(["dev/media/final/panelbuttons.psd"], async () => {
  await BuildSkin("skins/src/std.µcss.mjs");
}, { debounceMs: 1500 });

OpenDrafts("dev/media/final/panelbuttons.psd", { app: "affinity" });
```

Umgebungsvariablen: MU_AFFINITY_EXE, MU_DRAFT_APP. Details: docs/CONCEPT.md (D11).

4.11.1 Zwischenschritte raw → final (outputBase)

Standardmäßig schreiben alle Steps in das Skin-Ausgabeverzeichnis. Mit `outputBase: "project"` schreibt ein Step stattdessen relativ zum Projektstamm — damit lässt sich die Erzeugung von Zwischenergebnissen (z. B. Sequenz-Strips aus `dev/media/raw/...` nach `dev/media/final/...`) direkt im Manifest abbilden. Die Steps laufen in Manifest-Reihenfolge, ein nachfolgender `copy/copyFolder`-Step übernimmt das Ergebnis dann wie jedes andere Final-Asset in den Skin:

```
media: [
  // 1. Strip aus den Roh-Frames in den final-Baum generieren ...
  { sequenceStrip: "dev/media/raw/flyex/frames", outputFile:
"dev/media/final/general/gui/flyex/flyex.png", outputBase: "project" },
  // 2. ... und von dort wie gewohnt in den Skin kopieren.
  { copyFolder: "dev/media/final/general/gui/flyex", to: "imgs/general/gui/flyex",
filter: "\\.(png|json)$" }
]
```

Auch für diese Steps gilt der **inkrementelle Build**: Generiert wird nur, wenn das Ergebnis fehlt oder sich Konfiguration bzw. Quellen (mtime/Größe) geändert haben. Alternativ kann der raw→final-Schritt natürlich weiterhin als eigener Gulp-Task vor dem µCSS-Build laufen — `outputBase: "project"` ist der Weg, wenn alles in einer Manifest-Datei stehen soll.

5 microAU (μAU) — Sound-Atlas

Das Schwester-Modul **μAU** (`gulp-mu-au`) baut aus vielen kurzen Audiodateien einen **Sound-Atlas** — Audio-Gegenstück zu Sprite-Atlanten: ein kombinierter WAV/MP3-Blob plus JSON-Timing-Map für die Browser-Runtime (z. B. `μLib Sounds`).

Stand: nur **Build-Layer**. Referenzieren von Sounds *aus der CSS* (`μCSS-Sound-Direktive`) und Browser-Runtime/Sprachausgabe folgen später.

Engine: `gulp-mu-sound-atlas` (Dekodieren → Resampling → optional MP3). `μAU` ist der `μPS`-artige Wrapper mit `async Create` und inkrementellem Cache.

5.1 API-Übersicht

Export	Beschreibung
<code>SoundAtlasMaker</code>	Quellen sammeln → Atlas + JSON bauen (gecacht)
<code>ListAudioFiles</code>	Rekursiv <code>*.wav/*.mp3</code> auflisten
<code>CONVERT</code>	Konstanten für Samplerate, Samplegröße, Kanäle, Stereo

5.2 SoundAtlasMaker

```
import { SoundAtlasMaker } from "gulp-mu-au";

const result = await SoundAtlasMaker.Create({
  src: "dev/media/final/sounds",
  dataFile: "skins/std/snds/app.sounds.wav",
  jsonFile: "skins/std/snds/app.sounds.json"
});
// result => { skipped, sounds, dataFile, jsonFile }
```

Option	Default	Wirkung
<code>dataFile</code>	— (Pflicht)	Audio-Blob (<code>.wav</code> oder <code>.mp3</code>)
<code>jsonFile</code>	— (Pflicht)	JSON-Timing-Map
<code>sounds / src</code>	—	Explizite Liste bzw. rekursives Quellverzeichnis; mindestens eines erforderlich
<code>format</code>	aus Endung	"wav" oder "mp3"
<code>mp3KBitRate</code>	128	MP3-Bitrate
<code>sampleRate, sampleSize, channels, stereo</code>	<code>CONVERT.*_HIGHEST</code>	Ziel-Format; <code>*_HIGHEST/*_LOWEST</code> aus Eingaben ableiten
<code>cacheFile</code>	<code><dataFile>.cache.json</code>	Cache-Marker, oder <code>false</code>
<code>force</code>	<code>false</code>	Neubau erzwingen

5.2.1 Loop-Punkte im Dateinamen

`engine.ls.100.le.400.wav` → Sound-Name `engine`, Loop von Sample 100 bis 400. Der Basisname ohne `.ls....le....`-Suffix ist der Schlüssel in der Map.

5.2.2 JSON-Format

```
{ "sounds": { "click": [startSec, durationSec, loopStartSec, loopEndSec], ... } }
```

-1 für loopStart/loopEnd = kein Loop. Das Format wird von `μLib Sounds` direkt konsumiert.

5.2.3 MP3-Eingaben

Für **MP3-Quelldateien** benötigt die Engine im Konsumentenprojekt ggf. einen `patch-package-Patch` (siehe `gulp-mu-sound-atlas-README`). WAV-Eingaben und MP3-*Ausgabe* sind unbeeinträchtigt.

5.3 Anbindung an `μCSS (Manifest `sounds`)`

`μCSS` ruft `μAU` über einen optionalen `sounds`-Block im Skin-Manifest auf (parallel zu `media`):

```
export default DefineSkin({
  sounds: {
    src: "dev/media/final/sounds",
    dataFile: "snds/app.sounds.wav",
    jsonFile: "snds/app.sounds.json"
  },
  files: [ ... ]
});
```

Pfade in `dataFile/jsonFile` sind relativ zum Skin-Ausgabeverzeichnis; `src` relativ zum Projektstamm. Der Build-Report enthält `report.sounds[]` mit `skipped` und der Sound-Liste. Inkrementeller Cache wie bei `μPS-Media-Steps`.

Alternativ bleibt `copyFolder` für bereits gebaute Atlanten möglich.

5.3.1 Geplant: CSS-Sounds, Auto-Wiring & Handler-Overrides

Der **Build-Layer** (Atlas + JSON-Timing-Map inkl. Loop-Punkte pro Sound) ist umgesetzt. **Loop ja/nein** steht in `sounds[name]` — nicht in den Bindings. Bindings legen nur fest, **wann** abgespielt/gestoppt wird (`mode: oneshot` vs. `sustain`).

Pipeline (festgelegt): Build liefert **Daten** (`sounds + bindings[]` im JSON) — aus CSS und optional `soundTriggers.mjs` (läuft **nur** in Node). Runtime liefert **Verhalten**: zuerst `Sounds.installBindings` (vollautomatisch), optional danach `soundHandlers.mjs` (patcht Default-Handler im Browser — **ohne** JSON zu ändern, **ohne** Build-Ausführung). `handlers` ersetzt nicht `triggers`: neue Events → CSS/`triggers`; abweichende Reaktion → `handlers` oder App-JS.

Als nächste Stufen:

- 1. Deklarativ im CSS** — aurale Properties (`cue-before`, `cue-after`, `play-during`) und Direktive `-μ: Sound(...)` → Einträge in `bindings[]` (Sidecar im JSON).
- 2. Vollautomatisch (μLib)** — `Sounds.installBindings(json)` legt alle DOM-/Animations-Listener an; kein manuelles Event-JS nötig, wenn CSS + `triggers` reichen.
- 3. Manifest `soundTriggers.mjs`** (`sounds.triggers`) — Build-Zeit: `bindings[]` ergänzen (Helper-Selektoren, FlyEx-Animationen). **Nicht** `helpers.mjs`.
- 4. Manifest `soundHandlers.mjs`** (`sounds.handlers`, optional) — Runtime: Standard-Handler **wrappen/ersetzen** (Super-Aufruf auf die Default-Methode), z. B. für FlyEx-Klick-Logik neben Auto-Wiring.
- 5. App-JS** — weiterhin `Sounds.play/Sounds.stop` für freie Logik ohne Binding.

Details: `gulp-mu-au/docs/CONCEPT.md` (§5–§8, Abschnitt „Pipeline“). FlyEx-Demo heute: alles in `demo.js`; Ziel: Loops über `triggers` + Auto-Wiring, Sonderfälle über `handlers`.

6 microFT (μFT) — Icon-Font

μFT (`gulp-mu-ft`) erzeugt aus SVG-Glyphen einen **Icon-Font** — automatisiert wie IcoMoon, ohne Adobe/IcoMoon-Abhängigkeit. Ausgabe: Font-Dateien (SVG/TTF/EOT/WOFF/WOFF2), CSS-Klassen, IcoMoon-kompatible JSON, HTML-Übersicht.

Erfordert **Node ≥ 20.11**. Aufgesetzt auf `svgicons2svgfont`, `svg2ttf`, `ttf2woff(2)`, `ttf2eot`.

6.1 Glyphen-Benennung

`<name>-U0x<HEX>.svg`
z. B. `general-control-edit-U0xE900.svg` → Name "general-control-edit", Code U+E900

- **Verzeichnis** = Gruppen-ID (relativ unter `src`, z. B. `general` oder `medical/diagnosis`)
- Ohne gültiges `-U0x<HEX>` → übersprungen (Entwürfe)
- Doppelte Codepoints → Warnung, Konfliktdateien werden genannt

6.2 FontGenerator

```
import { FontGenerator } from "gulp-mu-ft";

const result = await FontGenerator.Create({
  fontName: "AppSymbol",
  src: "svg",
  outputDir: "fonts",
  groups: {
    general: { label: "General", description: "General UI controls.", order: 1 }
  },
  glyphs: {
    "general-control-edit": { description: "Button/icon to start editing." }
  }
});
// result => { skipped, glyphs, warnings, files }
```

Erzeugt u. a. `AppSymbol.css` (`.icon-<name>`), `AppSymbol.json`, `AppSymbol.html`.

Option	Default	Wirkung
<code>fontName</code> , <code>src</code> , <code>outputDir</code>	— (Pflicht)	Font-Name, SVG-Quellbaum, Ziel
<code>formats</code>	alle gängigen	Zu erzeugende Font-Endungen
<code>fontHeight</code>	1024	Em-Höhe
<code>normalize</code> , <code>centerHorizontally</code>	true	an <code>svgicons2svgfont</code>
<code>classPrefix</code>	"icon"	CSS-Klassen-Präfix
<code>fontUrlBase</code>	""	URL-Präfix in CSS/HTML
<code>css</code> , <code>json</code> , <code>html</code>	aktiv	Ausgabe steuern oder false
<code>groups</code> , <code>glyphs</code>	{}	Metadaten für HTML-Übersicht (nur Englisch)
<code>timestamp</code>	fest	Reproduzierbare Font-Zeitstempel
<code>cacheFile</code>	<code>.build-cache.json</code>	Inhalts-Signatur (SHA-1), zeitstempel-unabhängig
<code>force</code>	false	Neubau erzwingen

6.3 API-Bausteine

Export	Beschreibung
FontGenerator	Scan → Font bauen → CSS/JSON/HTML schreiben
ScanGlyphs	Verzeichnis scannen, Codepoints/Gruppen ableiten
BuildFontFormats, BuildFontCss, BuildIcoMoonJson, BuildOverviewHtml	Einzelne Build-Schritte
ListSvgFiles, ReadGlyphSvg	Low-Level-Helfer

6.4 Anbindung an µCSS

Manifest (geplant): optionaler Block `font` in Kapitel *Manifest-Referenz* — `src` (SVG-Verzeichnis), `include` (Einzel-SVGs), `outputDir`, Metadaten; baut über µFT vor der CSS-Kompilierung.

Heute: typischerweise als **eigener Gulp-Step** vor dem Skin-Build, danach **copyFolder** im Manifest:

```
media: [  
  { copyFolder: "dev/media/final/fonts", to: "fonts", filter: "\\.(woff2?|ttf|css)$" }  
]
```

Geplant zusätzlich: CSS-Direktive `-µ: Glyph("icons/edit.svg")` für Einzel-SVGs in Regeln (symmetrisch zu `Sprite()/Sound()`). Icon-Fonts werden in `.µ.css` per `@font-face` und Klassen referenziert (Variablen für Codepoints, z. B. `icon_pencil: "\\e91c"` in `vars`).

7 Vue und Komponenten-Co-Location

Komponenten-Frameworks wie Vue halten Styles üblicherweise in `<style scoped>` innerhalb der `.vue`-Datei. Das funktioniert, aber jede Komponente bekommt eigene `data-v-...`-Attribute, das CSS wächst schnell und Live-Bearbeitung in den Browser-DevTools wird über viele Scoped-Blöcke hinweg unhandlich.

μCSS verfolgt einen anderen Ansatz: **Co-Location ohne Vue-Verarbeitung der Styles**. Jede Komponente behält eine Style-Datei **neben** ihrer `.vue`-Datei, aber mit einer privaten Endung wie `*.π.css` (griechisches Pi — Vite und Vue ignorieren sie). Ein Haupt-Stylesheet sammelt alles zur Build-Zeit ein; μCSS bündelt daraus eine einzige, statische CSS-Datei.

7.1 Ablauf

1. Komponenten-Styles in `MyButton.π.css` neben `MyButton.vue` ablegen.
2. Im Haupteinstieg (z. B. `main.μ.css`) per Build-Time-Import einsammeln:

```
@import "components/**/*.π.css";
body { margin: 0; }
```

3. Regel-Merge im Manifest aktivieren, wenn viele Komponenten dieselben Selektoren nutzen (z. B. `.btn`):

```
export default DefineSkin({
  merge: { onConflict: "error" },
  files: [{ source: "main.μ.css", target: "app.css" }]
});
```

4. Im Vue-Template **normale Klassennamen** verwenden — kein `<style scoped>`, kein `data-v-...`. Der Browser lädt eine echte CSS-Datei; die DevTools bleiben voll nutzbar.

7.2 Namespaces und globale State-Klassen

Wenn zwei Komponenten `.card` definieren, Merge aktivieren (oben) oder Kollisionen mit `@μ-namespace` pro Komponentendatei vermeiden:

```
@μ-namespace MyButton;
.btn { padding: 10px; }
.btn:global(.is-active) { box-shadow: 0 0 2px #99bbff; }
```

Nur Klassen dieser Datei werden präfixiert (`.MyButton-btn`). Von JavaScript gesetzte State-Klassen (`.is-active`, geteilte Utilities) bleiben über `:global(...)` global.

7.3 Bestehende Vue-SFCs migrieren

Im Repository gibt es eine Migrationshilfe analog zum LESS-Konverter. Sie extrahiert `<style>`-Blöcke aus `.vue`-Dateien, schreibt die Sidecar-Datei `ComponentName.π.css`, setzt `@μ-namespace`, entfernt Scoped-`[data-v-...]`-Selektoren und löscht den `<style>`-Block aus der SFC (ein HTML-Kommentar verweist auf die Sidecar):

```
npx gulp convert:vue
# oder: VUE_IN=gulp-mu-css/examples/vue VUE_OUT=out npx gulp convert:vue
```

Beispielquellen liegen unter `gulp-mu-css/examples/vue/`. Das Tool schreibt zusätzlich `main.μ.css` mit `@import "**/*.π.css"`; und ein Manifest-Skelett inkl. `merge.onConflict: "error"`. Warnungen vor dem Einsatz prüfen — `lang="scss"`, CSS Modules und `v-bind()` in CSS erfordern manuelle Nacharbeit; `lang="less"` wird zuerst durch den LESS-Konverter geleitet.

8 Grundideen von µCSS

8.1 Das .µ.css-Format

Quell-Stylesheets heißen *.µ.css. Das Doppelsuffix sorgt dafür, dass Editoren die Dateien als normales CSS erkennen und das Syntax-Highlighting ohne weitere Konfiguration funktioniert. Für den Compiler ist die Endung irrelevant — das Manifest referenziert die Quellen explizit; wer das µ-Zeichen in Dateinamen vermeiden möchte, kann ebenso *.mu.css verwenden. Inhaltlich ist eine .µ.css-Datei Standard-CSS mit genau zwei Erweiterungspunkten — beide sind syntaktisch valides CSS, so dass Editoren und Linter normal funktionieren:

1. Werte-Interpolation µ(Ausdruck) — überall, wo ein CSS-Wert steht.

2. Direktiven -µ: Ausdruck; — als Deklaration innerhalb einer Regel.

Wer das µ-Zeichen nicht auf der Tastatur hat, verwendet die ASCII-Aliasse `mu(...)` und `-mu:` — beide Formen sind gleichwertig.

Anders als beim alten µCSS gibt es **keine Steuer-Regeln** (`::-µcss-init` usw.) mehr im Stylesheet: Alles, was die Kompilierung steuert (Variablen, Cursor-Definitionen, Medienerzeugung, Dateizuordnung), steht im Skin-Manifest — einer normalen JavaScript-Datei (siehe unten).

8.2 Werte-Interpolation µ(Ausdruck)

µ(...) ist aus CSS-Sicht eine unbekannte, aber valide Funktion. Beim Kompilieren wird der Inhalt als JavaScript-Ausdruck ausgewertet und das µ(...)-Vorkommen durch das Ergebnis ersetzt:

```
::selection {
  background-color: µ($.selectBaseBrkdOnDarkColor);
  color: µ($.selectBaseTextColor);
}
```

Mehrere Interpolationen pro Wert sind erlaubt (`padding: µ($.padY)px µ($.padX)px;`), ebenso Interpolationen in At-Rule-Parametern (z. B. `@media (max-width: µ($.breakpoint)px)`). Liefert ein Ausdruck `null` oder `undefined`, bricht die Kompilierung mit einer Fehlermeldung ab — so fallen Tippfehler in Variablennamen sofort auf.

Diese eine Erweiterung ersetzt die komplette `Set*`-Familie des alten µCSS (`SetColor`, `SetBackgroundColor`, `SetZIndex`, `SetWidth`, `AddProperty` für einfache Werte usw.): Die Property steht wieder an ihrem Platz, ein Platzhalter-Wert ist nicht mehr nötig.

8.3 Direktiven -µ: Ausdruck

Direktiven sind Deklarationen mit dem Property-Namen `-µ` (oder `-mu`). Ihr Wert ist ein einzelner JavaScript-Ausdruck, der mit Zugriff auf die umgebende Regel und das Dokument ausgeführt wird. Die Direktive selbst wird aus der Ausgabe entfernt:

```
div.panel.modal div.companylogo {
  -µ: Sprite("imgs/logos/company_logo.png");
  margin-left: auto;
}
*.cursor_zoom {
  -µ: Cursor("zoom");
}
div.content.glittery {
  -µ: Sprite("imgs/general/gui/glittery/glittery.png", { afterWork: GlitterySprite });
}
```

Das µ.-Präfix des alten µCSS entfällt — der Kontext ist implizit. Direktiven werden in Dokumentreihenfolge ausgewertet.

8.4 Das Skin-Manifest

Pro Skin (CSS-Thema) liegt im Quellverzeichnis eine Manifest-Datei `<skinname>.μcss.mjs` — normales JavaScript (ES6+), importierbar und testbar. Das Doppelsuffix `.μcss.mjs` (ASCII-Alternative `.mucss.mjs`) kennzeichnet die Datei eindeutig als `μCSS`-Skin-Manifest, damit sie nicht mit einem gewöhnlichen Modul oder Gulpfile verwechselt wird; der Skin-Name ergibt sich aus dem Teil davor (`std.μcss.mjs` → Skin `std`). Sie ersetzt die alte Steuerdatei `μ.std.css` vollständig:

```
// skins/src/std.μcss.mjs – Skin "std", Ziel: skins/std/
import { DefineSkin } from "gulp-mu-css";
import { GlitterySprite, FlyEx, Borders, TableBackgrounds } from "./helpers.mjs";

export default DefineSkin({
  vars: {
    baseBrgdColor: "#202020",
    selectBaseBrgdColor: "#007570",
    PanelZIndex: 9000
  },
  helpers: { GlitterySprite, FlyEx, Borders, TableBackgrounds },
  cursors: [
    { name: "zoom", fallback: "zoom-in", image: "imgs/general/gui/cursors/zoom.png",
    hotspot: [10, 8] }
  ],
  media: [
    { buttonsAndIcons: "dev/media/final/general/gui/panelbuttons.psd", layout: "std",
    outputDir: "imgs/general/gui/panelbuttons" },
    { copyFolder: "dev/media/final/fonts", to: "fonts" }
  ],
  imageFormat: "png",
  sprites: { file: "imgs/sprites.png", retina: true, preloadRule: true },
  files: [
    { source: "src.μ.css", target: "std.css" },
    { source: "src_tinymce.μ.css", target: "std_tinymce.css" }
  ]
});
```

Der Skin-Name ergibt sich aus dem Dateinamen des Manifests, das Ausgabeverzeichnis aus dem Skin-Namen. Die vollständige Feld-Referenz steht im Kapitel „Manifest-Referenz“.

8.5 JavaScript ohne Ersatzzeichen

Bei `μCSS` 1 mussten die Zeichen `{`, `}` und `;` in JavaScript-Anweisungen durch `«`, `»` und `;` ersetzt werden, um CSS-Syntaxkonflikte zu vermeiden. Dieses Muster entfällt in Version 2 ersatzlos:

- In `μ(...)` und `-μ: ...` stehen **einzelne Ausdrücke** — der Parser zählt Klammern und berücksichtigt Strings, so dass auch Objekt-Literale (`{ afterWork: ... }`) und verschachtelte Aufrufe problemlos funktionieren.
- **Mehrzeilige Logik** (Schleifen, Funktionsdefinitionen) gehört nicht ins Stylesheet, sondern in die Helper-Module des Manifests — echte `.mjs`-Dateien mit Syntax-Highlighting, Linting und Debugger.

9 Build-Ablauf und Gulp-Integration

Das alte `µCSS` wurde über ein modales Dialogfenster in Photoshop bedient. An dessen Stelle treten `BuildSkin` und `Gulp`: Der Build ist ein normaler, skriptbarer Node-Prozess — geeignet für Watch-Modus, CI und Automatisierung.

9.1 Verzeichniskonventionen

Für ein Manifest `skins/src/std.µcss.mjs` gilt (alle Pfade überschreibbar):

Pfad	Bedeutung
<code>skins/src/</code>	Quellverzeichnis: Manifeste, <code>.µ.css</code> -Dateien, Helper-Module
<code>skins/std/</code>	Ausgabeverzeichnis des Skins „std“: CSS, Bilder, Fonts, Sounds
<code>skins/std/.cache/build.json</code>	Build-Cache des Skins (Fingerprints, Atlas-Positionen)
Projektstamm	Zwei Ebenen über dem Manifest; Basis für media-Quellpfade wie <code>dev/media/...</code>

`BuildSkin(manifestPfad, optionen)` akzeptiert `{ outputDir, rootDir, force }` zum Übersteuern.

9.2 Kompilierungs-Ablauf

Ein `BuildSkin`-Lauf führt fünf Schritte aus:

- 1. media-Steps:** Alle Bilder werden erzeugt bzw. kopiert (`microPS`) — sie müssen existieren, bevor Atlas und Cursor-Prüfungen laufen.
- 2. Kompilierung:** Jede `files`-Quelle wird geparkt (`PostCSS`); Direktiven und Interpolationen werden in Dokumentreihenfolge ausgewertet. `Sprite()`-/`Cursor()`-Aufrufe registrieren zunächst nur ihre Bildreferenzen.
- 3. Sprite-Atlas:** Alle registrierten Bilder werden gepackt (inkl. `@2x`); danach werden die betroffenen Regeln umgeschrieben.
- 4. Hooks:** `afterWork`-Callbacks laufen mit Atlas-Ergebnis und AST-Zugriff.
- 5. Ausgabe:** Die kompilierten CSS-Dateien werden in das Skin-Verzeichnis geschrieben (optional minifiziert über `minify`), der Cache wird gespeichert.

9.3 Inkrementeller Build und Cache

Jeder Lauf erzeugt nur das neu, was sich tatsächlich geändert hat. Primärer Mechanismus sind Datei-Modifikations-Zeitstempel (`mtime` plus Dateigröße) — bei großen PSD-Quellen um Größenordnungen schneller als Inhalts-Hashes:

- **Generator-Steps** (`buttonsAndIcons`, `appIcons`, `sequenceStrip`) werden übersprungen, wenn ihre Konfiguration unverändert ist, alle Quelldateien unveränderte Fingerprints haben und alle Ausgabedateien noch existieren. PSD-Rendering ist der teuerste Posten des Builds — hier zahlt der Cache am meisten.
- `copy/copyFolder` arbeiten Make-artig: kopiert wird nur, wenn die Quelle neuer als das Ziel ist.

- **Der Sprite-Atlas** wird nur neu gepackt, wenn sich die Bildmenge oder ein Quellbild (inkl. @2x) geändert hat — sonst werden die gespeicherten Positionen wiederverwendet und nur die CSS-Regeln damit umgeschrieben.
- **Die CSS-Kompilierung** selbst ist billig und läuft im Zweifel immer.

Der Cache liegt pro Skin in `<outputDir>/cache/build.json` und trägt die Versionsnummern von `μCSS` und das Cache-Schema; bei Abweichung wird er verworfen (Vollbuild). `BuildSkin(manifest, { force: true })` oder das Löschen von `cache/` erzwingt den Vollbuild explizit.

9.4 Gulp-Tasks und Watch

Ein Task pro Skin genügt; `BuildSkin` ist re-entrant:

```
import gulp from "gulp";
import { BuildSkin } from "gulp-mu-css";

export async function SkinStd() {
  const report = await BuildSkin("skins/src/std.μcss.mjs");
  console.log(`${report.skin}: ${report.files.length} Dateien, Atlas $
{report.atlasSkipped ? "aus Cache" : "neu gepackt"}, ${report.duration} ms`);
}
export function SkinWatch() {
  gulp.watch(["skins/src/**/*.μ.css", "skins/src/*.mjs", "dev/media/final/**"], SkinStd);
}
```

Der Rückgabewert von `BuildSkin` ist ein Report mit `skin`, `outputDir`, `media` (Step-Ergebnisse mit `skipped-Flag`), `files`, `atlas`, `atlasSkipped`, `prunedSources` (gelöschte Sprite-Quell-URLs bei `sprites.pruneSources`), `keptSources` (geschützte Quellen bei `sprites.pruneKeep`), `minified` (`true`, wenn `minify` aktiv war) und `duration`.

10 Manifest-Referenz (DefineSkin)

`DefineSkin(konfiguration)` deklariert die Skin-Konfiguration (Default-Export des Manifests) und validiert die Grundstruktur. Alle Felder sind optional, sofern nicht anders angegeben.

10.1 vars

Objekt mit den Skin-Variablen — der Ersatz für `μ.$.*`. In `μ.css`-Ausdrücken als `$.name` zugreifbar, in Helper-Funktionen als `this.$.name`. Werte sind beliebige JavaScript-Werte (Strings, Zahlen, Objekte, auch berechnete).

```
vars: { baseBrgdColor: "#202020", PanelZIndex: 9000, icon_pencil: "\\e91c" }
```

10.2 helpers

Objekt mit benutzerdefinierten Funktionen (Makros und Hooks). Sie stehen in `μ.css`-Ausdrücken unter ihrem Namen zur Verfügung. Funktionen, die mit `function` (nicht als Arrow-Funktion) deklariert sind, erhalten beim Aufruf `this` = Auswertungs-Scope (siehe Kapitel „μ-Auswertungskontext“).

```
import { Borders, GlitterySprite } from "./helpers.mjs";  
// ...  
helpers: { Borders, GlitterySprite }
```

Abgrenzung: Helpers = **CSS zur Compile-Zeit**. Geplant: `sounds.triggers` (Build → JSON-Daten) und `sounds.handlers` (Runtime → Verhalten patchen, optional) — strikt getrennt; siehe Kapitel **microAU**, Abschnitt „Pipeline“.

10.3 cursors

Liste der Cursor-Definitionen — der Ersatz für `μ.DefCursor`:

Feld	Default	Bedeutung
<code>name</code>	— (Pflicht)	Name, unter dem der Cursor mit <code>Cursor("name")</code> verwendet wird.
<code>fallback</code>	<code>= name</code>	Standard-Cursor-Name (W3C) als Fallback.
<code>image</code>	<code>""</code>	Bild-URL relativ zum Skin-Verzeichnis; leer = Definition nur als Fallback-Mapping.
<code>hotspot</code>	<code>[0, 0]</code>	Klickpunkt <code>[x, y]</code> im Bild; <code>0,0</code> wird in der Ausgabe weggelassen.
<code>forceFallback</code>	<code>false</code>	Hängt den Fallback zusätzlich als letzte cursor-Deklaration an.

Cursor-Bilder werden automatisch in die Preload-Liste aufgenommen.

10.4 media

Liste der Medien-Steps; sie laufen in der angegebenen Reihenfolge vor der Kompilierung. Der Step-Typ ergibt sich aus dem Schlüsselfeld:

Step	Pflichtfelder	Weitere Felder	Wirkung
buttonsAndIcons	Quell-PSD, layout, outputDir	retina (Default true), format, mode, setPattern	Button-/Icon-Serien aus einem Entwurfsdokument rendern (microPS ButtonAndIconCreator). mode: "topLayerSets" schaltet auf „ein Bild pro Top-Untergruppe“ um (Legacy-CreateByTopLayerSets, ohne icons-Gruppe — für Logos, Animationsframes, Emoticons); setPattern (Regex-String) filtert dabei die exportierten Gruppen.
appIcons	Quell-PSD/PNG	outputDir, profiles, layout, background, appName, shortName, themeColor	App-Icons/Favicons profilbasiert generieren (microPS ApplconMaker).
sequenceStrip	Quelle (Ordner oder DSD-Bild), outputFile	retina (Default true), writeMapFile (Default true), format	Horizontalen Animations-Strip plus JSON-Frame-Daten erzeugen (microPS SequenceStrip).
copy	Quelldatei	to (Zielordner, Default Skin-Wurzel)	Einzeldatei kopieren, wenn die Quelle neuer ist.
copyFolder	Quellordner	to (Default = Ordnername), filter (Regex-String, z. B. "\\.(woff2? ttf)\$")	Ordner rekursiv kopieren (Make-artig), optional gefiltert.

Quellpfade sind relativ zum Projektstamm, Zielpfade relativ zum Skin-Verzeichnis. Jeder Step akzeptiert zusätzlich outputBase: "skin" (Default) oder "project": Mit "project" ist der Zielpfad relativ zum Projektstamm — für Zwischenschritte wie raw → final (siehe Kapitel „Automatische Bilderzeugung“).

10.5 imageFormat

Globaler Bildformat-Schalter: "png" (Default) oder "webp". **Wirkungsbereich:**

- **Bilderzeugende media-Steps** (buttonsAndIcons, appIcons*, sequenceStrip) und der **Sprite-Atlas** (sofern sprites.format nicht gesetzt ist, siehe unten).
- Einzelne Steps können das Format per eigenem format-Feld übersteuern.
- Direkt referenzierte Bestandsbilder (url(...) in den Quellen, copy-/copyFolder-Steps) bleiben unangetastet — ein copyFolder-filter kopiert nur, was er durchlässt. Schließt der filter die aktive imageFormat-Endung aus (z. B. "\\.(png|json)\$" bei imageFormat: "webp"),

gibt der Build eine **Warnung** aus, statt die gerade erzeugten Dateien stillschweigend zu verwerfen.

`*\*appIcons` erzeugt plattformspezifische Icon-Sätze (PNG, teils ICO) und **ignoriert** `imageFormat` **bewusst** — App-/Favicon-Formate sind festgelegt.*

10.6 sprites

Optionen des Sprite-Atlas — der Ersatz für `μ.options.sprites.*`:

Feld	Default	Bedeutung
<code>file</code>	<code>"imgs/sprites.png"</code>	Atlas-Datei (URL, wie sie in der CSS erscheint).
<code>format</code>	(entfällt)	Atlas-Ausgabeformat (" <code>png</code> "/" <code>webp</code> "), unabhängig von <code>imageFormat</code> . Gesetzt, überschreibt es die aus <code>imageFormat</code> abgeleitete Endung; sonst gilt <code>imageFormat</code> . So lässt sich allein der Atlas auf WebP umstellen (<code>sprites: { file: "imgs/sprites.png", format: "webp" }</code>), ohne die Generator-Steps anzufassen — die <code>Sprite()</code> -Quellbilder dürfen weiterhin PNG sein.
<code>retina</code>	<code>true</code>	Erzeugt zusätzlich <code><name>@2x</code> aus den <code>@2x</code> -Quellbildern (müssen neben den 1x-Quellen liegen).
<code>padding</code>	<code>0</code>	Abstand in Pixeln zwischen den Sprites.
<code>preloadRule</code>	<code>false</code>	Erzeugt die <code>div.csspreload</code> -Regel im ersten Stylesheet.
<code>include</code>	(entfällt)	Einzelbilder oder Verzeichnisse (relativ zum Skin-Output), die ohne CSS-Regel in den Atlas sollen (z. B. Preload-only oder JS-Assets).
<code>pruneSources</code>	<code>false</code>	Nach erfolgreichem Atlas-Build die Quell-PNGs/WebPs löschen , die nur noch im Atlas liegen (1x und <code>@2x</code>). Ersatz für das alte <code>gulp-mu-spritereducer</code> -Verhalten — opt-in für Deploy/Release-Builds. Der Atlas selbst wird nie gelöscht. Der Build-Report enthält <code>prunedSources[]</code> .
<code>pruneKeep</code>	<code>[]</code>	Nur mit <code>pruneSources: true</code> : skin-relative Dateien oder

Feld	Default	Bedeutung
		Verzeichnisse , die vom Trim ausgenommen bleiben. Abgleich über die 1x-URL jedes Sprites (exakte Datei oder Verzeichnis-Präfix); 1x und @2x bleiben erhalten. Beispiel: ["imgs/general/glittery", "imgs/x/y.png"]. Der Build-Report enthält keptSources[].

Auflösung der Sprite()-Quellbilder: Der in `Sprite("...")` referenzierte Pfad wird formatunabhängig aufgelöst — zuerst der literale Pfad, dann derselbe Name mit einer der unterstützten Raster-Endungen (png, webp). Liegt ein generiertes Quellbild also als PNG vor, obwohl die CSS-Referenz .webp nennt (oder umgekehrt), bricht der Build nicht ab. Ein Fehler entsteht nur, wenn **keine** Variante existiert. Die @2x-Variante muss dieselbe Endung wie das aufgelöste 1x-Bild haben.

10.7 minify

Optionales Top-Level-Feld für Deploy-/Release-Builds: Ist es gesetzt, wird jede erzeugte `files[].target-CSS` beim Schreiben durch einen Minifier geleitet.

Wert	Wirkung
false / entfällt	Keine Minifizierung; Zeilenumbrüche und Einrückung bleiben erhalten.
true	[<code>uglifycss</code>](https://www.npmjs.com/package/uglifycss) mit Defaults { <code>maxLineLen: 1000</code> , <code>uglyComments: true</code> } (Lieferumfang von <code>μCSS</code>).
Objekt	<code>uglifycss</code> mit diesen Optionen über den Defaults.
Funktion (<code>css</code>) => <code>css</code>	Eigener Minifier (beliebige Engine), ohne zusätzliche Abhängigkeit.

`uglifycss` ist konservativ: Es entfernt nur Whitespace und Kommentare, **sortiert oder merged keine Regeln** — die Ausgabe bleibt semantisch gleichwertig zur unminifizierten CSS.

Typisches Deploy-Manifest (Trim + Minify):

```
minify: true,
sprites: {
  file: "imgs/sprites.png",
  retina: true,
  pruneSources: true,
  pruneKeep: ["imgs/general/glittery"]
}
```

10.8 sounds

Optionaler Block für die **μAU**-Bridge — baut einen Sound-Atlas vor der CSS-Kompilierung (siehe Kapitel *microAU*):

Feld	Bedeutung
src	Quellverzeichnis (rekursiv *.wav/*.mp3), relativ

Feld	Bedeutung
	zum Projektstamm
sounds	Alternative: explizite Dateiliste (zusammen mit src möglich)
dataFile	Ausgabe-Audio-Blob, relativ zum Skin-Verzeichnis
jsonFile	Ausgabe-JSON-Timing-Map, relativ zum Skin-Verzeichnis
format, mp3KBitRate, sampleRate, ...	Werden an SoundAtlasMaker durchgereicht (Defaults wie in µAU)
triggers	*(geplant)* Pfad zu soundTriggers.mjs — nur Build-Zeit: bindings[] ergänzen (Node, kein Browser)
handlers	*(geplant)* Pfad zu soundHandlers.mjs — nur Runtime: Default-Handler wrappen/ersetzen (Browser, JSON unverändert)
force	Neubau erzwingen

Der Build-Report enthält sounds[] mit skipped, sounds (Namen) und Pfaden. Ohne sounds-Block: Sound-Dateien per copyFolder ins Skin kopieren.

10.9 font

Optionaler Block für die **µFT**-Bridge — baut einen Icon-Font aus SVG-Glyphen vor der CSS-Kompilierung (siehe Kapitel *microFT*). Symmetrie zu sounds und sprites: **Verzeichnis und/oder Einzeldateien** registrieren Glyphen im Font, auch ohne CSS-Referenz.

Stand: Die Manifest-Bridge ist **geplant** (noch nicht in BuildSkin umgesetzt). Üblich heute: FontGenerator.Create() als eigener Gulp-Step, danach copyFolder (siehe unten).

```
font: {
  fontName: "AppSymbol",
  src: "dev/media/svg", // rekursives SVG-Verzeichnis, relativ zum
Projektstamm
  include: ["dev/media/svg/extra/special-U0xE950.svg"], // *(geplant)* zusätzliche
Einzel-SVGs
  outputDir: "fonts", // relativ zum Skin-Ausgabeverzeichnis
  fontUrlBase: "../fonts/", // URL-Präfix in der erzeugten CSS
  groups: {
    general: { label: "General", description: "General UI controls.", order: 1 }
  },
  glyphs: {
    "general-control-edit": { description: "Button/icon to start editing." }
  }
}
```

Mehrere Fonts: font: [{ fontName: "AppSymbol", ... }, { fontName: "MedicalSymbol", ... }] — analog zu sounds als Array.

Feld	Bedeutung
fontName	Font-Familiennamen (Pflicht) — Basis für Dateinamen (AppSymbol.woff2, ...)
src	Quellverzeichnis mit SVG-Glyphen (rekursiv),

Feld	Bedeutung
	relativ zum Projektstamm
<code>include</code>	*(geplant)* zusätzliche Einzel-SVG-Dateien (relativ zum Projektstamm), kombinierbar mit <code>src</code>
<code>outputDir</code>	Zielverzeichnis im Skin (Font-Dateien, CSS, JSON, HTML-Übersicht)
<code>formats</code>	Zu erzeugende Endungen (Default: <code>svg</code> , <code>ttf</code> , <code>eot</code> , <code>woff</code> , <code>woff2</code>)
<code>fontHeight</code> , <code>normalize</code> , <code>centerHorizontally</code> , <code>classPrefix</code> , <code>fontUrlBase</code>	Werden an <code>FontGenerator</code> durchgereicht (Defaults wie in <code>µFT</code>)
<code>css</code> , <code>json</code> , <code>html</code>	Ausgabe steuern oder <code>false</code> zum Unterdrücken
<code>groups</code> , <code>glyphs</code>	Metadaten für die HTML-Übersicht (nur Englisch)
<code>force</code>	Neubau erzwingen

Glyphen-Benennung: `<name>-U0x<HEX>.svg` — Codepoint aus dem Dateinamen, Gruppen-ID aus dem Verzeichnis unter `src` (Details im Kapitel **microFT**).

Geplant — CSS-Einzelreferenz: Direktive `-µ: Glyph("icons/edit.svg")` registriert ein SVG zusätzlich im Font und schreibt die Regel mit `content/font-family` um (Gegenstück zu `sprites.include / Sound()` — siehe `gulp-mu-au/docs/CONCEPT.md §1`).

Heute ohne Manifest-Bridge:

```
// 1. Gulp-Task (vor BuildSkin): FontGenerator.Create({ fontName, src, outputDir:
"dev/media/final/fonts" })
media: [
  { copyFolder: "dev/media/final/fonts", to: "fonts", filter: "\\.(woff2?|ttf|css)$" }
]
```

Icon-Fonts in `.µ.css` per `@font-face` und Klassen; Codepoints oft als Escape in `vars` (z. B. `icon_pencil: "\\e91c"`).

10.10 files

Liste der zu kompilierenden Stylesheets — der Ersatz für `µ.DependentCSSFile`. `source` ist relativ zum Quellverzeichnis, `target` relativ zum Skin-Verzeichnis; beide Felder sind Pflicht:

```
files: [
  { source: "src.µ.css", target: "std.css" },
  { source: "src_tinymce.µ.css", target: "std_tinymce.css" }
]
```

Alle Dateien eines Skins teilen sich Variablen, Helpers und den Sprite-Atlas. Die Preload-Regel landet im ersten Stylesheet.

11 μ -Auswertungskontext (Referenz)

Jeder $\mu(\dots)$ -Ausdruck und jede $-\mu$ -Direktive wird in einem Scope ausgewertet, der die folgenden Bindungen enthält. Er entspricht dem alten globalen μ -Objekt — nur ohne Präfix.

11.1 Das \$-Objekt

\$ enthält die vars des Manifests. Lesen und Schreiben ist möglich; Änderungen gelten für den weiteren Verlauf der Kompilierung (Dokumentreihenfolge, über alle files eines Skins hinweg).

```
div.box { z-index:  $\mu$ ($.PanelZIndex + 1); }
```

11.2 Farb- und Wert-Funktionen

Funktion	Beschreibung
Lighten(color, step, model = "hsl")	Hellet eine Farbe relativ auf (positiver step) oder dunkelt sie ab (negativer step): $L' = \text{clamp}(L + L \cdot \text{step})$. Das Modell "hsl" ist bitgenau zum alten μ CSS; "oklch" skaliert wahrnehmungsgleichmäßig. Alpha bleibt erhalten.
Alpha(color, alpha)	Ersetzt den Alpha-Kanal einer Farbe. alpha nach den Regeln aus „Arbeiten mit Farben“.
MixColors(color1, color2)	Kanalweiser Mittelwert zweier Farben (inklusive Alpha).
AlphaValue(alpha)	Wandelt eine Alpha-Angabe in ein Byte (0–255) um.
ParseColor(color)	Parst eine CSS-Farbe in die interne 32-Bit-Darstellung 0xAARRGGBB.
FormatColor(color, alphaDecimals = 3)	Serialisiert die interne Darstellung zurück zu CSS (#rrggbb bzw. rgba(...)).
PxUnit(value)	Zahlen werden zu "<n>px", alles andere (z. B. calc()-Strings) wird unverändert durchgereicht.

11.3 Regel- und Dokument-Methoden

Innerhalb von Direktiven (und Helper-Funktionen mit this-Bindung) stehen zusätzlich die Manipulations-Methoden der umgebenden Regel bereit:

Methode / Eigenschaft	Beschreibung
AddProperty(name, value, important?)	Hängt eine Deklaration an die Regel an (Duplikate erlaubt — für Fallback-Ketten wie mehrfache background-image).
ChangeProperty(name, value, important?)	Ändert alle Deklarationen der Property bzw. legt sie an, wenn sie fehlt.
RemoveProperty(name)	Entfernt alle Deklarationen der Property.
AddRule(selector)	Fügt eine neue Regel am Dokumentende an; Rückgabe ist die Regel (mit denselben Methoden).

Methode / Eigenschaft	Beschreibung
<code>InsertRule(selector)</code>	Fügt eine neue Regel direkt hinter der aktuellen Regel ein; aufeinanderfolgende Aufrufe behalten ihre Reihenfolge. Ersatz für das alte <code>AddBlock(n, μ.elementNo)</code> .
<code>rule</code>	Die umgebende Regel als <code>CssRule</code> -Objekt (<code>rule.selector</code> , <code>rule.GetProperty(...)</code> , ...).
<code>document</code>	Das gesamte Stylesheet als <code>CssDocument</code> — z. B. für Pfad-Adressierung: <code>document.FindRule("@keyframes glittery", "from")</code> . Ersatz für die alten <code>RememberBlocks</code> .

11.4 Sprite()

Registriert das Bild der Regel für den Sprite-Atlas (nur als Direktive):

```
-μ: Sprite(url, optionen?);
```

Parameter / Option	Default	Beschreibung
<code>url</code>	— (Pflicht)	Bild-URL relativ zum Skin-Verzeichnis.
<code>offsetWidth</code>	0	Wird zur berechneten <code>width</code> addiert.
<code>offsetHeight</code>	0	Wird zur berechneten <code>height</code> addiert.
<code>offsetPosX</code>	0	Wird zur <code>background-position-X</code> -Koordinate addiert.
<code>offsetPosY</code>	0	Wird zur <code>background-position-Y</code> -Koordinate addiert.
<code>afterWork</code>	—	Hook-Funktion, läuft nach der Atlas-Auflösung (siehe unten).

Beim Atlas-Auflösen werden `background-image (url(...) plus image-set(...) bei Retina)`, `background-repeat`, `background-position`, `width` und `height` der Regel gesetzt; vorhandene Deklarationen dieser Properties werden ersetzt.

11.5 Cursor()

Wendet eine Cursor-Definition aus dem Manifest an — als Direktive (`-μ: Cursor("zoom");`, schreibt die `cursor`-Deklarationen der Regel um) oder als Wertform (`cursor: μ(Cursor("zoom"))`); liefert nur den `url(...) x y, fallback-String`). Unbekannte Namen werden unverändert durchgereicht — Standard-Cursor wie `pointer` funktionieren so ohne Definition.

11.6 Helper-Funktionen und this-Bindung

Helper aus dem Manifest, die mit `function` deklariert sind, werden mit `this` = Auswertungs-Scope aufgerufen. Damit lassen sich Makros im Stil der alten `μ.$`-Funktionen schreiben — nur als normales, testbares JavaScript:

```
// helpers.mjs
```



```
import { Lighten } from "gulp-mu-css";

export function Borders(_baseColor, _pixelWidth, _topLighten, _rightLighten,
_bottomLighten, _leftLighten) {
  this.AddProperty("border-top", `${_pixelWidth}px solid ${Lighten(_baseColor,
_topLighten)}`);
  this.AddProperty("border-right", `${_pixelWidth}px solid ${Lighten(_baseColor,
_rightLighten)}`);
  this.AddProperty("border-bottom", `${_pixelWidth}px solid ${Lighten(_baseColor,
_bottomLighten)}`);
  this.AddProperty("border-left", `${_pixelWidth}px solid ${Lighten(_baseColor,
_leftLighten)}`);
}
```

Über `this` sind alle Scope-Bindungen erreichbar: `this.AddProperty(...)`, `this.InsertRule(...)`, `this.rule`, `this.document` und `this.$`. Die Bindung bleibt auch erhalten, wenn ein Helper als `afterWork`-Wert an `Sprite()` übergeben wird. Arrow-Functions haben kein eigenes `this` und eignen sich daher nur für Helpers ohne Regel-Zugriff.

Portierte Referenz-Makros (`Borders`, `TableBackgrounds`, `GlitterySprite`, `FlyEx`, `FlyExUtils`) liegen im μ CSS-Repository unter `test/fixtures/reference-macros.mjs` (Demo-/Test-Fixture, nicht Teil der npm-API).

11.7 afterWork-Hooks

Der `afterWork`-Hook einer `Sprite`-Registrierung läuft, nachdem der Atlas gepackt und die Regel umgeschrieben wurde. Er erhält einen Kontext mit allen Ergebnisdaten:

Feld	Beschreibung
<code>rule</code>	Die umgeschriebene Regel (<code>CssRule</code>).
<code>document</code>	Das Stylesheet (<code>CssDocument</code>).
<code>url</code>	Die registrierte Bild-URL.
<code>baseDir</code>	Skin-Verzeichnis (zum Auflösen von Nachbar-Dateien, z. B. <code>.json</code> -Maps).
<code>sprite</code>	<code>{ x, y, width, height }</code> — Position und Größe im Atlas.
<code>atlas</code>	<code>{ file, retinaFile, width, height }</code> — die Atlas-Dateien und -Gesamtgröße.

Typischer Einsatz: Animations-Keyframes aus den Frame-Daten eines `sequenceStrip` generieren (siehe `GlitterySprite` in den Referenz-Makros).

12 Arbeiten mit Farben

12.1 Farben definieren

Die interne Darstellung einer Farbe ist ein vorzeichenloser 32-Bit-Integer der Form `0xAARRGGBB` (Alpha, Rot, Grün, Blau; je 8 Bit). In allen Farb-Funktionen sind folgende Eingabeformen möglich:

- **32-Bit-Integer** wie die interne Darstellung, z. B. `0xffff0000` für deckendes Rot.
- **#-Notation** mit zwei Hex-Ziffern pro Kanal, z. B. `"#00ff00"` (Alpha wird auf deckend gesetzt).
- **Kurze #-Notation** mit einer Hex-Ziffer pro Kanal, z. B. `"#0f0"`.
- **Erweiterte #-Notation** mit Alpha-Kanal als vierte Komponente, z. B. `"#0000ff80"` für halbtransparentes Blau.
- **rgb()-Notation**, absolut oder prozentual, z. B. `"rgb(255,0,0)"` oder `"rgb(100%,0%,0%)"`.
- **rgba()-Notation** mit Float-Alpha, z. B. `"rgba(255,0,0,0.5)"`.
- **CSS-Farbnamen** (W3C CSS Color Module, erweiterte Liste), z. B. `"red"`, `"teal"`, `"rebeccapurple"` sowie `"transparent"`.

Werte außerhalb des gültigen Bereichs werden begrenzt (geclippt).

12.2 Alpha-Werte

Alpha-Angaben (z. B. für `Alpha(color, a)`) sind in mehreren Formen möglich:

- **Float** zwischen `0.0` (transparent) und `1.0` (deckend).
- **Integer** zwischen `0` und `255`.
- **Hex-String**, z. B. `"0x80"`.
- **Prozent-String** von `"0%"` bis `"100%"`.
- **Schlüsselwörter** `"transparent"` (0), `"translucent"` (128) und `"opaque"` (255).

12.3 Farbberechnungen

Die Funktionen `Lighten`, `Alpha` und `MixColors` (Kapitel „μ-Auswertungskontext“) decken die typischen Berechnungen ab. Deckende Ergebnisse werden als `#rrggbb` ausgegeben, transparente als `rgba(r,g,b,a)` mit drei Alpha-Nachkommastellen — identisch zum alten μCSS.

```
div.menu {
  background-color: μ(Alpha($.baseBrgdColor, 0.9));
  border-top: 1px solid μ(Lighten($.baseBrgdColor, 0.3));
  border-bottom: 1px solid μ(Lighten($.baseBrgdColor, -0.3));
}
```

Für neue Skins empfiehlt sich das `"oklch"`-Modell von `Lighten`: Es verändert die wahrgenommene Helligkeit gleichmäßig über alle Farbtöne, während das Legacy-Modell `"hsl"` bei gesättigten Farben sichtbar springen kann.

13 Node-API

Neben dem Manifest-Workflow ist jede Schicht von μ CSS auch direkt als Node-API nutzbar — etwa für eigene Gulp-Transformationen oder Tests.

Export	Beschreibung
<code>CompileMcCss(source, options)</code>	Kompiliert <code>.μ.css</code> -Quelltext zu einem <code>CssDocument</code> . Optionen: <code>vars</code> , <code>helpers</code> , <code>from</code> (Dateiname für Fehlermeldungen), <code>context</code> (gemeinsamer <code>MuContext</code>), <code>sprites</code> (<code>SpriteManager</code>), <code>cursors</code> (<code>CursorManager</code>).
<code>CssDocument</code> / <code>CssRule</code>	JSON-fähiger Wrapper über den PostCSS-AST: <code>FromFile/FromString</code> , <code>FindRule(...pfad)</code> , <code>FindRules(selektorOderRegex)</code> , <code>AddRule</code> , <code>GetProperty/AddProperty/ChangeProperty/RemoveProperty</code> , <code>ToCss()</code> , <code>ToFile()</code> , <code>ToJson()/FromJson()</code> . Für Spezialfälle ist der rohe PostCSS-AST über <code>document.root</code> zugänglich.
<code>SpriteManager</code>	Sprite-Registrierung und Atlas-Auflösung: <code>new SpriteManager({ baseDir, atlasFile, retina, padding, preloadRule, preload, writeMapFile, pruneSources, pruneKeep })</code> , <code>Register(rule, url, optionen)</code> , <code>await Resolve(document)</code> (setzt <code>lastPruned</code> mit <code>pruned[]/kept[]</code>).
<code>CursorManager</code>	Cursor-Definitionen: <code>new CursorManager(definitionen, { baseDir, preload })</code> , <code>Apply(rule, name)</code> , <code>Value(name)</code> .
<code>PreloadRegistry</code>	Sammelt Bild-URLs und erzeugt die <code>div.csspreload</code> -Regel.
<code>DefineSkin(config)</code> / <code>BuildSkin(manifest, options)</code>	Manifest-Deklaration und Skin-Build (siehe Kapitel „Build-Ablauf“).
<code>MuContext</code>	Auswertungskontext für eigene Pipelines: <code>new MuContext({ vars, helpers })</code> , <code>Evaluate(Quelltext, extraScope)</code> .
<code>Lighten</code> , <code>Alpha</code> , <code>AlphaValue</code> , <code>MixColors</code> , <code>ParseColor</code> , <code>FormatColor</code> , <code>PxUnit</code>	Die Farb- und Wert-Funktionen als direkte Importe.

Beispiel einer JSON-basierten Gulp-Manipulation:

```
import { CssDocument } from "gulp-mu-css";

const doc = await CssDocument.FromFile("skins/src/src. $\mu$ .css");
doc.FindRules(/^div\.panel/).forEach(_rule => _rule.AddProperty("outline", "none"));
doc.FindRule("@keyframes glittery", "from").ChangeProperty("background-position-x", "-128px");
const json = doc.ToJson();
await doc.ToFile("out/std.css");
```

14 Fehlerdiagnostik

Build-Fehler nennen immer den Verursacher samt Quellbezug:

- **Inline-JavaScript** ($\mu(\dots)$ -Interpolationen und $-\mu$:-Direktiven): Fehler werden als PostCSS-Fehler mit Datei, Zeile, Spalte und Quelltext-Ausschnitt gemeldet. Bei mehreren $\mu(\dots)$ in einem Wert wird der fehlschlagende Ausdruck genannt. JavaScript-Syntaxfehler erscheinen als `invalid JavaScript expression "<Quelltext>" (...)`.

```
CssSyntaxError: skins/src/std.μ.css:2:2: microCSS: μ(NopeFn(1)): NopeFn is not defined
```

```
1 | div.b {  
> 2 |     border: 1px solid μ(NopeFn(1));  
   |           ^  
3 | }
```

- **Fehlende Sprite-Bilder**: Vor dem Packen des Atlas wird die Existenz aller Quellbilder (inklusive @2x bei `retina: true`) geprüft. Alle fehlenden Dateien werden gesammelt in einem Fehler gemeldet — jeweils mit URL, aufgelöstem Pfad und der referenzierenden Regel:

```
SpriteManager: 2 sprite image(s) not found:  
- "imgs/nope.png" -> C:\...\skins\std\imgs\nope.png - selector "div.bad1" (skin.μ.css:2)  
- "imgs/also nope.png" -> C:\...\skins\std\imgs\also nope.png - selector "div.bad2"  
(skin.μ.css:3)
```

- **afterWork-Hooks**: Fehler im Hook werden mit Sprite-URL und Regel-Quellposition ummantelt; der Original-Fehler bleibt als `cause` erhalten.
- **Fehlende Cursor-Bilder** erzeugen nur eine Warnung (einmal pro Cursor), da die CSS über den Fallback-Cursor funktionsfähig bleibt.
- **media-Steps**: Fehler nennen Step-Nummer und -Typ, z. B. `BuildSkin: media step 3 of 7 (buttonsAndIcons: "dev/media/buttons.psd") failed: ....` Fehlende Quellen werden vor der Ausführung geprüft: `copy/copyFolder` und Generator-Steps melden den aufgelösten Pfad statt eines rohen Dateisystemfehlers — bei `copyFolder` mit dem Hinweis, dass vermutlich der erzeugende Schritt (z. B. Sequenzbild-Generierung nach `dev/media/final/...`) nicht gelaufen ist.
- **files-Einträge**: Fehlende Quelldateien werden mit Eintrag und aufgelöstem Pfad gemeldet, bevor kompiliert wird.

15 Migration von µCSS

Für Bestandsprojekte existiert ein Konverter-Werkzeug (`tools/convert-mucss.mjs` im µCSS-Repository), das die alte Syntax mechanisch übersetzt:

Alt (µCSS 1)	Neu (µCSS 2)
<code>-µcss: µ.Cursor("wait");</code> plus <code>cursor: wait;</code>	<code>cursor: µ(Cursor("wait"));</code>
<code>-µcss: µ.SetBackgroundColor(µ.\$.x);</code> plus Platzhalter	<code>background-color: µ(\$.x);</code>
<code>-µcss: µ.AddProperty("border", "1px solid " + µ.Lighten(...));</code>	<code>border: 1px solid µ(Lighten(...));</code>
<code>-µcss: µ.Sprite("p.png");</code>	<code>-µ: Sprite("p.png");</code>
<code>-µcss: µ.SetRememberBlock("n");</code>	entfällt — Pfad-Adressierung: <code>document.FindRule("@keyframes x", "from")</code>
<code>-µcss: //µ.X(...)</code> (deaktiviert)	<code>/* -µ: X(...) */</code>
<code>µ.\$.name = wert</code> (in <code>µ.std.css</code>)	vars-Eintrag im Manifest
<code>µ.DefCursor(...)</code>	cursors-Eintrag im Manifest
<code>µ.plugins.* / FileCopy</code>	media-Einträge im Manifest
<code>µ.\$.Fn = function(...)</code> «...»	exportierte Funktion in <code>helpers.mjs</code>

Die «»i-Funktionskörper werden zu normalem JavaScript zurückübersetzt und landen als Startpunkt in `helpers.mjs`; manuelle Nacharbeit ist hier eingeplant.

raw → final automatisch: Das alte µCSS kopierte nur die bereits fertig erzeugten final-Ordner in den Skin (z. B. `flyex`, `glittery`); die Strips selbst entstanden außerhalb (`SpriteTools`). Der Konverter erkennt solche `CopyFolder2Skin`-Aufrufe auf `dev/media/final/<...>/<name>` und stellt — falls eine passende Quelle `dev/media/raw/<name>/imgs` existiert — automatisch den passenden `sequenceStrip`-Step mit `outputBase: "project"` davor: eine **nummerierte PNG-Frame-Sequenz** (z. B. `glittery`) wird zu einem Ordner-Strip, **einzeln benannte Bilder** (z. B. die DSD-Bilder `flyex.png/flyexutils.png`) je zu einem DSD-Strip. So baut die neue Pipeline `final` reproduzierbar aus `raw` und der nachfolgende `copyFolder` übernimmt das Ergebnis in den Skin.

Der Konverter wurde an einem vollständigen Legacy-µCSS-1-Bestand validiert: Ein Vergleichswerkzeug (`tools/compare-legacy-skin.mjs`, nur Repository) baut den konvertierten Skin und vergleicht ihn regel- und property-weise gegen die alte kompilierte Ausgabe — Ergebnis: 2 951 Regeln, 0 unerwartete Differenzen (53 dokumentierte Drift-Fälle, z. B. nach dem letzten µCSS-Lauf editierte Quellen).

Bewusst **nicht** übernommen wurden aus dem alten µCSS:

- **Vendor-Prefixes** (`-webkit-`/`-moz-`/`-ms-`--Duplikate): Alle genutzten Features sind seit Jahren unprefixed verfügbar. Vendor-spezifische Selektoren ohne Standard-Pendant (z. B. `::-webkit-scrollbar`) werden natürlich unverändert durchgereicht.
- **Photoshop-Dialogfenster** im Build: Interaktive Parameter gehören in das Manifest oder in Umgebungsvariablen.
- **FTP-Synchronisation**: Dafür gibt es heute etablierte Werkzeuge (CI/CD, `rsync`, `gulp`-Plugins).
- **Die Ersatzzeichen** «»i: siehe Kapitel „Grundideen“.

16 Versionshistorie

Datum	Version	Anmerkungen
2013	1.0	Ursprüngliches µCSS als Adobe-Photoshop-Script: -µcss:-Direktiven, Sprite-Atlas, PSD-Plugins, Steuerung über µ.std.css.
2026-06	2.0.0	Vollständige Node.js-Neuimplementierung (npm-Paket gulp-mu-css, ohne Adobe-Abhängigkeit): Core-Pipeline (M1), Sprites & Cursor (M2), Manifest & Build mit inkrementellem Cache (M3), Hooks & Makros (M4), Legacy-Skin-Migration mit Konverter und Abnahmetest (M5), Handbuch (M6).
2026-06	2.1.0	Atlas-Format unabhängig vom globalen imageFormat über sprites.format; formatunabhängige Auflösung der Sprite()-Quellbilder (png/webp); Warnung, wenn ein copyFolder-filter das aktive Bildformat ausschließt.
2026-06	2.2.0	Normalisierung veralteter Gradient-Richtungen beim Kompilieren: linear-gradient(top bottom left right, ...) (ohne Prefix ungültig) wird auf die Standard-to ...-Form gehoben.
2026-06	2.2.1	Handbuch-Aktualisierung (Versionshistorie ergänzt, Deckblatt-Version korrigiert); keine funktionalen Änderungen gegenüber 2.2.0.
2026-06	2.2.2	Zweisprachiges Handbuch (microCSS-de.pdf / microCSS-en.pdf) und zweisprachige READMEs (Englisch zuerst) für gulp-mu-css und gulp-mu-ps; Build-Tooling für mehrsprachige Handbücher erweitert. Keine funktionalen Code-Änderungen.
2026-06	2.2.3	Fix: Überschriften tragen

Datum	Version	Anmerkungen
		explizite Outline-Level, damit das automatisch aktualisierte Inhaltsverzeichnis der PDF-Handbücher (DE/EN) befüllt wird (war zuvor leer). Reine Tooling-/Doku-Korrektur.
2026-06	2.2.4	Sound-Atlas-Anbindung (sounds-Block im Manifest über <code>μAU</code>); <code>sprites.include</code> für Einzelbilder/Verzeichnisse im Sprite-Atlas ohne CSS-Regel.
2026-06	2.3.0	Build-Zeit- <code>@import</code> -Bundling (Glob, rekursiv, Vue-Co-Location); opt-in Regel-Merge (<code>merge</code> , <code>@μ-override</code> , <code>onConflict</code>); Namespace-Modus (<code>@μ-namespace</code> , <code>:global()</code>); Build-Typ-/Varianten-Filter (<code>buildFilter</code> , über <code>gulp-mu-build-filter</code>). Handbuch: Kapitel Vue-Co-Location. Nur im Repository: Migrationshilfen <code>tools/convert-vue.mjs</code> , <code>tools/convert-less.mjs</code> . README/Paket-Metadaten: GitHub-Monorepo-Links. Neue Laufzeit-Abhängigkeiten: <code>postcss-selector-parser</code> , <code>gulp-mu-build-filter</code> .
2026-06	2.4.x	Handbuch *microPS* : Tabellen zu Fülloptionen (inkl. neu Kontur/Glanz), Blend-Modi, Stilübertragung vs. Ebenenverknüpfung; Referenz-PSD um <code>stroke*/satin*</code> -Layouts erweitert. μPS : <code>stroke</code> und <code>satin</code> in <code>Effects.mjs</code> mit Adobe-PNG-Regression (<code>ReferenceRender.test.mjs</code>).
2026-06	2.4.2	Handbuch *microAU*/*microFT* : Sound-Pipeline (<code>triggers/handlers</code>), Manifest-Abschnitt <code>font</code> ; aktualisierte PDFs (DE/EN). μPS 1.3.0 (Kontur/Glanz), μAU 0.1.3 (Sound-Konzept-Doku).
2026-06	2.5.0	Manifest <code>minify</code> (CSS-

Datum	Version	Anmerkungen
		Minifizierung via uglifycss, optional eigene Funktion); sprites.pruneKeep schützt Quellbilder vor pruneSources; Build-Report keptSources[]/minified. Neue Laufzeit-Abhängigkeit uglifycss.
2026-06	2.5.1	Handbuch/PDFs (DE/EN): Kapitel minify, sprites.pruneKeep, Build-Report; Versionshistorie; keine funktionalen Änderungen gegenüber 2.5.0.

17 Rechtliches

Diese Software wird unter der MIT-Lizenz veröffentlicht und ist für freie und kommerzielle Nutzung freigegeben. Weder Dongleware noch der Autor haften für Schäden, die durch die Nutzung dieser Software entstehen. Die Nutzung erfolgt auf eigenes Risiko; sichern Sie Ihre Arbeit, bevor Sie die Werkzeuge einsetzen.

Autor: Meinolf Amekudzi.

17.1 MIT-Lizenz (Originaltext)

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

17.2 Marken

Photoshop ist eine eingetragene Marke von Adobe Inc.; Affinity Photo ist eine Marke von Serif (Europe) Ltd. Die Nennung von Namen und Produkten dient ausschließlich der Information und stellt keinen Missbrauch der jeweiligen Handelsnamen oder Marken dar.

17.3 Verwendete Bibliotheken

μCSS und μPS nutzen Open-Source-Bibliotheken Dritter, insbesondere PostCSS (CSS-Parser, MIT-Lizenz), sharp (Bildverarbeitung, Apache-2.0-Lizenz), ag-psd (PSD-Leser, MIT-Lizenz) und uglifycss (CSS-Minifizierung, MIT-Lizenz). Der Bin-Packer des Sprite-Atlas basiert auf node-bin-packing (©2011 Jake Gordon und Mitwirkende, MIT-Lizenz).